Incorporating Deep Descriptive Clustering into the DECCS Algorithm

for Enhanced Interpretability and Performance on AwA2 and aPY Datasets

Mehdi Benabed

A thesis submitted in partial fulfillment
of the requirements for the degree of
Master of Science in Computer Science

Faculty of Computer Science
University of Vienna
Austria
August 31, 2026

Abstract

This thesis presents DDECCS, a framework that combines consensus-based clustering with interpretable cluster descriptions for attribute-annotated image datasets. The approach integrates two complementary methods: DECCS (Deep Clustering with Consensus Representations), which improves clustering robustness through ensemble agreement, and DDC (Deep Descriptive Clustering), which generates concise, human-readable cluster explanations via Integer Linear Programming (ILP).

Our pipeline extracts frozen ResNet-101 features, applies consensus clustering from a heterogeneous ensemble of four algorithms (K-means, Spectral Clustering, Gaussian Mixture Models, and Agglomerative Clustering), and generates orthogonal ILP descriptions—which penalize attribute overlap across clusters to keep explanations distinctive—using the datasets' semantic attributes. We introduce an adaptive ILP coverage parameter (α) that extends DDC's formulation to datasets with sparse binary attributes, where DDC's fixed $\alpha = 8$ cannot be satisfied and the description step would otherwise fail.

We evaluate on two established datasets: Animals with Attributes 2 (AwA2, 50 classes, 85 attributes) and Attribute Pascal and Yahoo (aPY, 15 and 32 classes, 64 attributes). On AwA2, frozen ResNet-101 features already cluster well (K-means NMI = 0.869), and DDECCS reaches NMI = 0.871—approaching DDC's published 0.896 without task-specific training; on this dataset the consensus step mainly improves accuracy and reduces variance rather than raising NMI. On aPY, DDC's trained model remains stronger (published NMI = 0.863), but our consensus step still improves substantially over a single-algorithm K-means baseline—by 15.6% relative NMI on the 32-class task (0.534 $\rightarrow$ 0.618)—while reducing run-to-run NMI variance by a factor of two to six depending on the dataset. The post-hoc ILP descriptions are semantically coherent, producing explanations such as "orange, stripes, claws, muscle" for a tiger cluster and "Pedal, Handlebars, Text, Plastic" for a bicycle cluster.

Because no implementation of DDC is publicly available, we additionally rebuild it from its published specification and evaluate it against matched baselines on both datasets under identical features and splits. The controlled comparison shows that DDC's self-generated pairwise constraints reduce to class supervision on per-class-attribute data, and that its description-to-clustering channel does not improve clustering on either dataset—inert on AwA2 and consistently harmful on aPY.

The thesis also documents extensive experiments with alternative approaches—including autoencoder-based training and DDC-style optimization on frozen features—together with a code-level diagnosis of why they failed, providing practical insights into the challenges of multi-objective deep clustering.

Zusammenfassung

Deep-Clustering-Verfahren lernen gemeinsam eine Merkmalsrepräsentation und eine Clusterzuordnung, stützen sich dabei jedoch typischerweise auf ein einzelnes Clustering-Ziel und liefern keine für Menschen verständliche Erklärung dafür, was die einzelnen Cluster voneinander unterscheidet. Diese Arbeit stellt DDECCS (Deep Descriptive Clustering with Consensus Representations) vor, ein Verfahren, das die interpretierbaren Cluster-Beschreibungen von Deep Descriptive Clustering (DDC) mit der ensemblebasierten Robustheit des DECCS-Konsens-Clusterings verbindet. Die Pipeline extrahiert 2048-dimensionale Merkmale aus einem eingefrorenen, auf ImageNet vortrainierten ResNet-101, clustert diese entweder mit K-Means oder mit einem Konsens-Ensemble aus vier Basisalgorithmen und erzeugt anschließend mittels ganzzahliger linearer Optimierung (ILP) für jeden Cluster eine kompakte, möglichst trennscharfe Menge semantischer Attribute.

Da das feste $\alpha = 8$ von DDC bei dünn besetzten binären Attributen zu einer unlösbaren ILP führt, führen wir ein adaptives α ein, das proportional zur mittleren Attributdichte skaliert und die Lösbarkeit auf solchen Datensätzen wiederherstellt. Wir evaluieren das Verfahren auf den attributannotierten Bilddatensätzen AwA2 und aPY. Auf AwA2 erreicht das Konsens-Clustering eine vergleichbare Clustering-Güte wie K-Means (NMI = 0,871) bei deutlich höherer Stabilität, während es auf dem 32-Klassen-aPY-Datensatz eine relative Verbesserung der NMI um 15,6 % gegenüber K-Means erzielt. Das ursprüngliche, end-to-end trainierte DDC bleibt auf aPY stärker (publizierte NMI = 0,863), da es seine Merkmale gezielt für die Clustering-Aufgabe verfeinert.

Ein zentrales negatives Ergebnis dieser Arbeit ist, dass K-Means auf den eingefrorenen ResNet-Merkmalen jede von uns trainierte Repräsentation übertraf: Die vortrainierten Merkmale kodieren bereits hinreichend klassentrennende Information, und unsere Trainingsläufe verschlechterten diese, statt sie zu verbessern. Eine nachträgliche Analyse des Trainingscodes ergab zudem Implementierungsfehler in den trainierten Varianten, sodass dieses Ergebnis unsere Implementierungen betrifft und die Frage offen lässt, ob ein vollständig korrekt umgesetztes Training die Merkmale verbessern könnte. Die erzeugten ILP-Beschreibungen sind kompakt und trennscharf – ein Tiger-Cluster wird etwa durch „orange, gestreift, Krallen, muskulös“ beschrieben – und das Beschreibungsmodul beeinflusst die Clusterzuordnung nicht, da es nachgelagert arbeitet. Insgesamt zeigt DDECCS, dass robustes Konsens-Clustering und interpretierbare Cluster-Beschreibungen ohne Einbußen bei der Clustering-Güte kombiniert werden können.

Contents

1. Introduction

Clustering complex image data is increasingly important across domains such as healthcare, scientific discovery, and content organization. Modern deep learning produces powerful representations for this task, but the resulting groupings are opaque: they come with no account of what sets one cluster apart from another. This thesis addresses that gap by combining robust consensus clustering with automatically generated, human-readable cluster descriptions. The remainder of this chapter motivates the problem, states our research questions and contributions, and outlines the structure of the thesis.

1.1. Motivation

This opacity is not incidental. Deep representations are optimized for downstream task performance, not for human inspection, so a clustering can be accurate yet offer no account of the features that distinguish one group from another [LeCun et al., 2015, Ren et al., 2024]. This lack of transparency is a critical limitation in domains where understanding the semantic meaning of clusters is as important as the clustering itself [Guidotti et al., 2018].

Two recent frameworks address complementary aspects of this challenge. Deep Clustering with Consensus Representations (DECCS) [Miklautz et al., 2022] improves clustering robustness through ensemble consensus, combining multiple clustering algorithms to find stable cluster boundaries, but produces no explanations. Deep Descriptive Clustering (DDC) [Zhang and Davidson, 2021] generates human-readable cluster explanations using semantic attributes and Integer Linear Programming (ILP)—producing concise, attribute-based descriptions of what each cluster represents—but relies on a single clustering algorithm without ensemble consensus. Combining the robustness of the former with the interpretability of the latter is the central idea of this thesis: a unified pipeline, DDECCS, that produces robust, interpretable clusters on attribute-annotated image datasets. We evaluate on two datasets used in the DDC literature: Animals with Attributes 2 (AwA2) and Attribute Pascal and Yahoo (aPY).

1.2. Research Objectives

This thesis addresses three research questions:

RQ1: How can DDC's ILP-based description mechanism be integrated into the DECCS consensus clustering framework?

RQ2: Does combining consensus clustering with ILP-based descriptions improve clustering performance or stability compared to using K-means alone?

RQ3: How does the integrated approach perform on the AwA2 and aPY datasets relative to DDC's published results?

1.3. Contributions

This thesis makes the following contributions:

- **The DDECCS framework**, which couples DECCS consensus clustering with DDC's post-hoc ILP cluster descriptions in a single pipeline built on frozen ResNet-101 features.
- **An adaptive ILP coverage parameter** that extends DDC's description formulation to datasets with sparse binary attributes, where DDC's fixed coverage requirement cannot otherwise be met.
- **A complete aPY preprocessing pipeline**, including identification and filtering of DDC's 15-class evaluation subset from the raw annotations.
- **A systematic empirical evaluation** across two datasets, four clustering modes, and ten random seeds, together with documented negative results from alternative approaches that informed the final design.
- **A controlled reproduction of the original DDC.** No public implementation exists. We rebuild the method from its published specification and evaluate it against matched baselines on both datasets, establishing that its self-generated constraints reduce to class supervision on per-class-attribute data, that its description-to-clustering channel does not improve clustering on either dataset, and that its published coverage parameter is unreachable on aPY.

1.4. Thesis Structure

The remainder of the thesis is organized as follows. **Chapter 2 (Related Work)** surveys deep clustering, consensus clustering, and explainable clustering, and analyses the DECCS and DDC frameworks that form our foundation. **Chapter 3 (Methodology)** formalizes the problem and presents the DDECCS pipeline: feature extraction, the clustering ensemble, ILP description generation, and adaptive parameter scaling, with the rationale for each design choice. **Chapter 4 (Experiments)** describes the experimental setup, datasets, and evaluation metrics, then reports quantitative results, comparison with DDC's published numbers, and qualitative analysis of the generated descriptions. **Chapter 5 (A Controlled Comparison with the Original DDC)** reports a reproduction of DDC from its published specification, evaluated against matched baselines on both datasets, and draws out what the comparison implies for DDECCS. **Chapter 6 (Discussion)** interprets the results, examines the trade-offs and why the trained alternatives underperformed, and revisits the research questions. **Chapter 7 (Conclusion)** summarizes the contributions, states the limitations and scope, distils the notable insights, and outlines directions for future work.

2. Related Work

This chapter surveys the research areas on which this thesis builds. We first review deep clustering and its dominant single-algorithm paradigm (Section 2.1), then consensus clustering and the DECCS framework that motivates our robustness goal (Section 2.2). We then turn to explainable and descriptive clustering, including the DDC framework that motivates our interpretability goal (Section 2.3). Finally, we summarize the capabilities of these methods and position our work against the gap none of them fills (Section 2.4).

2.1. Deep Clustering

Deep clustering jointly learns a feature representation and a cluster assignment, replacing the hand-crafted features used by classical algorithms. Deep Embedded Clustering (DEC) [Xie et al., 2016] is the seminal method: it pre-trains a stacked autoencoder, then refines the embedding by minimizing the Kullback–Leibler divergence between soft cluster assignments and a sharpened target distribution. DEC established the template of alternating between representation learning and cluster refinement that most later methods follow.

Subsequent work extended DEC along several directions. Improved DEC (IDEC) [Guo et al., 2017] retains the autoencoder's reconstruction term during clustering to preserve local structure. Deep Adaptive Clustering (DAC) [Chang et al., 2017] reformulates image clustering as a binary pairwise-classification problem. JULE [Yang et al., 2016] jointly learns representations and assignments in a recurrent agglomerative framework, and DeepCluster [Caron et al., 2018] alternates between k-means on learned features and using the resulting assignments as pseudo-labels. Deep embedded cluster trees [Mautz et al., 2020] extend the paradigm to hierarchical clustering. Comprehensive surveys of the field are provided by Min et al. [2018] and Zhou et al. [2025].

The common limitation across these methods is twofold. First, each is tied to a *single* clustering objective—typically k-means-style or spectral—so its quality depends on how well that objective's assumptions match the data. Second, the learned representations are opaque: they yield cluster assignments but no human-readable account of what distinguishes one cluster from another. The two families of work we review next address these two limitations independently.

2.2. Consensus Clustering

Consensus (or ensemble) clustering improves robustness by combining several clusterings into a single partition, so that the result does not depend on any one algorithm's assumptions. Classical consensus methods build a co-association matrix from multiple base partitions and then re-cluster it [Strehl and Ghosh, 2002, Fred and Jain, 2005];

broader surveys are given by Vega-Pons and Ruiz-Shulcloper [2011]. Their main limitation is that they operate on fixed input features or on the partitions alone, and are typically restricted to linear relationships—they do not learn a new representation from the raw data.

Deep Clustering with Consensus Representations (DECCS) [Miklautz et al., 2022] removes this limitation by learning a non-linear consensus representation with an autoencoder. The encoder is trained so that a heterogeneous ensemble of clustering algorithms (for example k-means, spectral clustering, and Gaussian mixtures) increasingly *agree* with one another when applied to the embedded data. Concretely, the encoder parameters Θ are optimized to maximize the average pairwise agreement of the ensemble members,

$$f_\Theta = c \sum_{i=1}^{|\mathcal{E}|} \sum_{j>i}^{|\mathcal{E}|} \mathrm{NMI}(e_i(\mathrm{enc}_\Theta(X)),\, e_j(\mathrm{enc}_\Theta(X)))\,,$$

where X is the input data, enc_Θ is the encoder, $\mathcal{E} = \{e_1, \ldots\}$ is the set of clustering algorithms, NMI is the normalized mutual information between two clusterings, and $c = 2/(|\mathcal{E}|^2 - |\mathcal{E}|)$ turns the sum into an average over the $|\mathcal{E}|(|\mathcal{E}| - 1)/2$ pairs. One round of DECCS, illustrated in Figure 2.1, embeds the data, applies the ensemble, trains classifiers to predict each member's assignments, and updates the encoder to increase agreement.

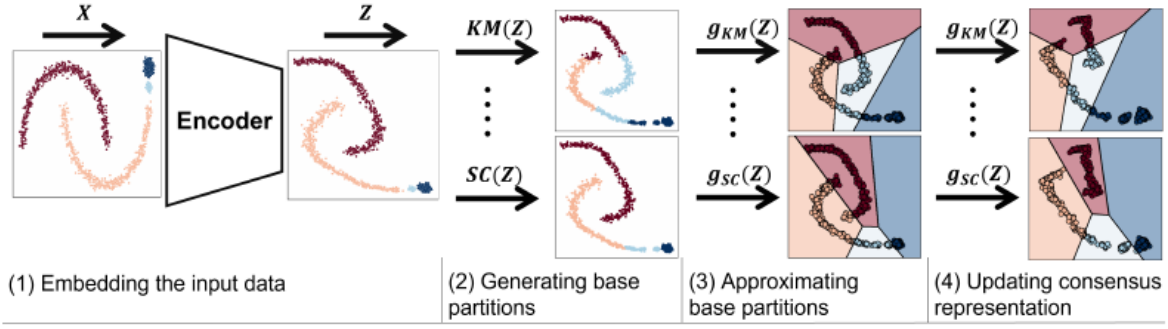

Figure 2.1: One round of DECCS [Miklautz et al., 2022]. (1) The encoder embeds data points X. (2) Clusterings are generated by applying ensemble members $\mathcal{E}$ to the embedding Z. (3) Classifiers g_i are trained to predict the cluster labels π_i from Z. (4) Z is updated by minimizing the loss $\mathcal{L}$.

The strength of DECCS is robustness: by maximizing agreement across diverse algorithms, it learns representations that are stable and reproducible rather than dependent on a single algorithm's inductive bias. Its limitation, for our purposes, is that the consensus is purely statistical—it produces no explanation of why instances are grouped together, and the learned representation has no semantic grounding.

2.3. Explainable and Descriptive Clustering

Explainable clustering methods fall into two broad families. *Explainable-by-design* methods [Bertsimas et al., 2021, Moshkovitz et al., 2020] build the clustering directly from interpretable features, for example by constructing a small decision tree whose splits both define and explain the clusters. Their strength is that the explanation is

exact and intrinsic; their weakness is that they require interpretable input features, which rules out complex data such as images whose raw pixels are not meaningful to a human.

Post-processing methods [Davidson et al., 2018, Sambaturu et al., 2020] instead take an existing clustering and attach explanations after the fact, typically using a separate set of semantic tags. They are algorithm-agnostic and can therefore explain the output of any clustering method, but because the explanation step is decoupled from clustering, the quality of the explanation depends entirely on how well the original clustering happens to align with the available tags. Instance-level methods such as LIME [Ribeiro et al., 2016] explain individual predictions with local surrogate models but do not characterize entire clusters.

Deep Descriptive Clustering (DDC) [Zhang and Davidson, 2021] bridges these families for complex data. DDC clusters by maximizing the mutual information between the inputs and the induced cluster labels, and simultaneously generates a cluster-level explanation by solving an Integer Linear Programming (ILP) problem that selects, for each cluster, a small set of semantic attributes. The ILP enforces *orthogonality* through an upper bound on how much each attribute's coverage may be shared across clusters, so that the descriptions for different clusters stay concise and minimally overlapping. A self-generated pairwise loss then feeds the explanation back into the clustering objective, encouraging the two modules to agree. The framework is illustrated in Figure 2.2.

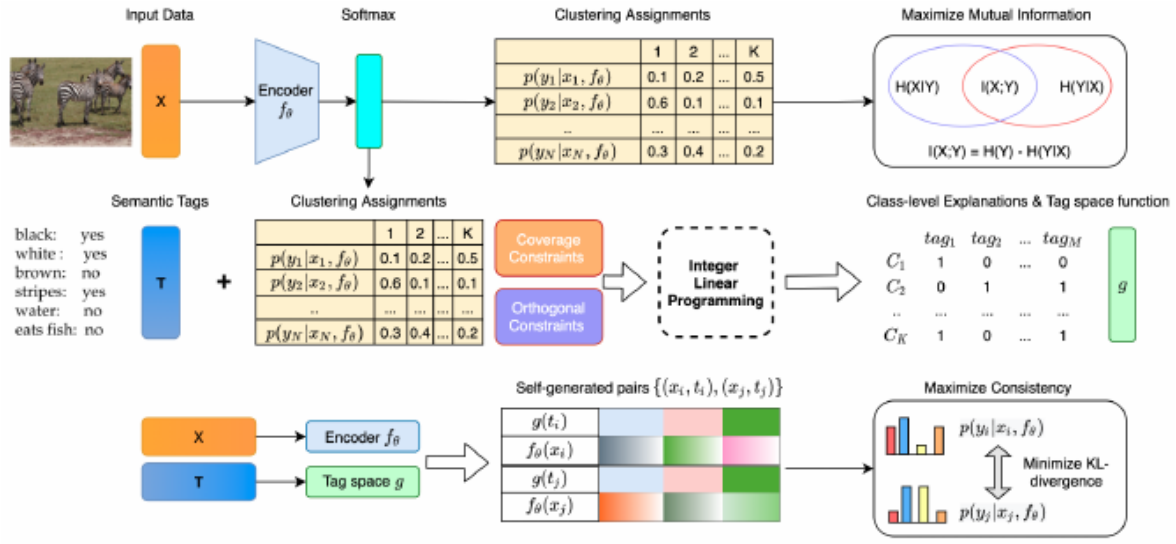

Figure 2.2: The DDC framework [Zhang and Davidson, 2021], comprising a clustering objective (mutual information maximization), a symbolic explanation objective (ILP-based tag selection), and a self-generated pairwise objective that maximizes consistency between the clustering and explanation modules.

The strength of DDC is interpretability: it is, to our knowledge, the first deep clustering method to produce concise, orthogonal cluster-level descriptions for complex image data. Its limitation, for our purposes, is that it relies on a single clustering objective and therefore does not benefit from the robustness that an ensemble provides. No implementation of DDC is publicly available; Chapter 5 reports a reproduction built from the published specification and evaluated against matched baselines on both datasets.

2.4. Summary and Research Gap

Table 2.1 summarizes the capabilities of the methods reviewed above. Two observations stand out. Consensus clustering (DECCS) delivers robustness but no interpretability, while descriptive clustering (DDC) delivers interpretability but uses a single clustering algorithm. No prior method provides both ensemble-based robustness and semantic cluster descriptions for complex image data.

Table 2.1: Capability comparison of the clustering methods reviewed in this chapter. DDECCS denotes the framework proposed in this thesis.

Capability	**DEC**	**DECCS**	**DDC**	**DDECCS**
Ensemble / consensus clustering	✗	✓	✗	✓
Uses semantic attributes	✗	✗	✓	✓
Cluster-level descriptions	✗	✗	✓	✓

DDECCS targets exactly this gap, combining DECCS-style consensus with DDC-style descriptions so that the final clusters are both robust and interpretable. The next chapter develops the pipeline in detail.

3. Methodology

This chapter presents the methodology of our integrated approach, which we term DDECCS (Deep Descriptive Clustering with Consensus Representations). The approach combines pretrained deep feature extraction with consensus-based ensemble clustering and interpretable cluster descriptions via Integer Linear Programming (ILP). The pipeline operates in four stages—feature extraction, clustering, description generation, and evaluation—which we present in turn, before situating the design relative to DECCS and DDC and justifying its main choices. Unlike approaches that jointly train feature representations and clustering objectives—which can suffer from loss-balancing instability, as we discuss in Section 3.6—our method decouples feature extraction from clustering. This leverages the strong visual representations already learned by modern pretrained networks while allowing each downstream component to operate independently and reliably.

Terminology. The DDC paper uses the term "tags" to refer to semantic properties of instances (e.g., "furry," "has wheels"). The dataset literature (AwA2, aPY) uses "attributes" for the same concept. Throughout this thesis, we use "attributes" as the primary term and note that DDC's "tags" are synonymous.

3.1. Problem Definition

Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{t}_i)\}_{i=1}^{N}$, where $\mathbf{x}_i$ is an image and $\mathbf{t}_i \in [0,1]^M$ is its associated semantic attribute vector over M attributes, our goal is to produce two outputs:

1. A partition of the data into K clusters $\{C_1, \ldots, C_K\}$ that aligns with the underlying semantic categories.
2. For each cluster C_k, a concise description $D_k \subset \{1, \ldots, M\}$: a small subset of attributes that characterizes the cluster. The descriptions are *orthogonal*, meaning the ILP discourages any attribute from being reused across many clusters, so that descriptions remain distinctive across clusters.

The clustering is assessed by its agreement with ground-truth class labels, and the descriptions by how well they cover their clusters and how distinctive they are; the specific metrics are defined in Chapter 4.

3.2. Pipeline Architecture

Our pipeline consists of four stages, illustrated in Figure 3.1:

1. **Feature Extraction**: Frozen ResNet-101 pretrained on ImageNet extracts 2048-dimensional feature vectors from input images.
2. **Clustering**: Either K-means (baseline) or DECCS consensus ensemble assigns images to clusters.
3. **Description Generation**: DDC's ILP formulation generates concise, orthogonal cluster descriptions using semantic attributes.
4. **Evaluation**: Standard clustering metrics (NMI, ACC, ARI, Silhouette) and interpretability metrics (TC, ITF) assess the results.

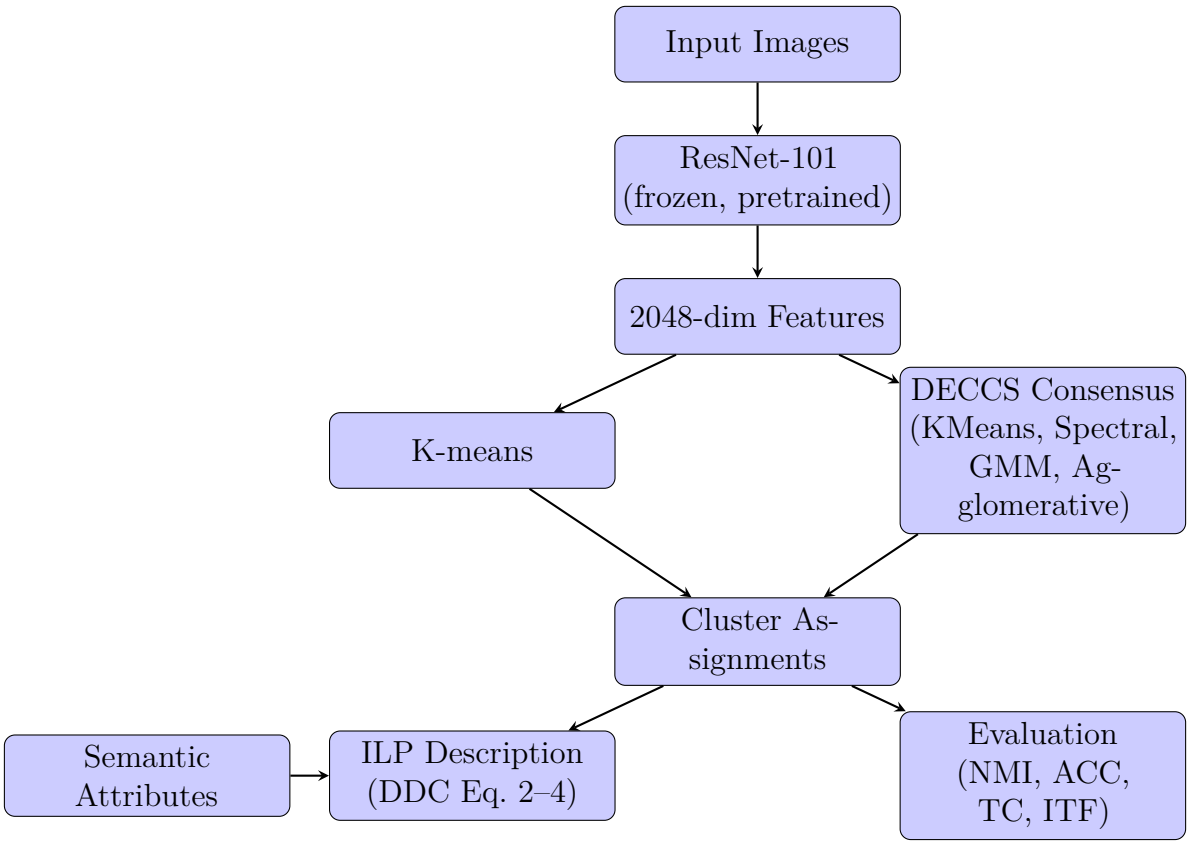

Figure 3.1: DDECCS pipeline architecture. Images are processed through a frozen ResNet-101 backbone to extract features, which are then clustered using either K-means or DECCS consensus. Cluster descriptions are generated via ILP using semantic attributes.

Algorithm 1 summarizes the complete DDECCS pipeline.

Algorithm 1 DDECCS Pipeline

Require: Image dataset $\mathcal{X}$, attribute matrix T, number of clusters K, mode $\in \{\texttt{kmeans}, \texttt{deccs}, \texttt{ddc}, \texttt{ddeccs}\}$
Ensure: Cluster assignments $\mathbf{y}$, cluster descriptions $\mathbf{D}$ (if ILP mode)

Step 1: Feature Extraction
Load frozen ResNet-101 (pretrained on ImageNet, final layer removed)
$F \leftarrow \emptyset$
for each image $x_i \in \mathcal{X}$ **do**
    Resize x_i to 224×224, normalize with ImageNet mean/std
    $\mathbf{f}_i \leftarrow \text{ResNet-101}(x_i) \in \mathbb{R}^{2048}$ ▷ Global average pooling output
    $F \leftarrow F \cup \{\mathbf{f}_i\}$
end for
Cache F to disk for subsequent runs

Step 2: Clustering
if mode $\in$ {kmeans, ddc} **then**
    $\mathbf{y} \leftarrow \text{KMeans}(F, K, n\_init = 10)$ ▷ raw features; see Sec. 4.4.4
else if mode $\in$ {deccs, ddeccs} **then**
    $F_{\text{std}} \leftarrow \text{Standardize}(F)$ ▷ zero mean, unit variance
    $\pi_1 \leftarrow \text{KMeans}(F_{\text{std}}, K)$
    $\pi_2 \leftarrow \text{SpectralClustering}(F_{\text{std}}, K, k\text{-NN affinity}, k = 15)$
    $F_{\text{pca}} \leftarrow \text{PCA}(F_{\text{std}}, d = \min(256, \dim, \max(64, N/2K)))$ ▷ For GMM numerical stability
    $\pi_3 \leftarrow \text{GMM}(F_{\text{pca}}, K, \text{full covariance})$
    $\pi_4 \leftarrow \text{Agglomerative}(F_{\text{std}}, K, \text{Ward linkage})$
    Build sparse co-association matrix over k-NN pairs of F_{std} ($k = 20$):
        $B_{ij} \leftarrow \sum_{m=1}^{4} \mathbb{1}[\pi_m(i) = \pi_m(j)]$ for $j \in \mathcal{N}_k(i)$, $A \leftarrow \frac{1}{4}(B + B^\top)$
    $\mathbf{y} \leftarrow \text{SpectralClustering}(A, K)$
end if

Step 3: ILP Description Generation (only if mode $\in \{\texttt{ddc}, \texttt{ddeccs}\}$)
Compute adaptive $\alpha \leftarrow \text{clip}(\text{round}(16 \cdot \bar{d}), 2, 8)$ where $\bar{d}$ is mean attribute density
for each cluster $i = 1, \ldots, K$ **do**
    for each attribute $j = 1, \ldots, M$ **do**
        $Q_{ij} \leftarrow$ mean of attribute j across instances in cluster i
    end for
end for
Solve ILP: minimize $\sum_{i,j} W_{ij}$ subject to:
  Coverage: $\sum_j W_{ij} \cdot Q_{ij} \geq \alpha \quad \forall i$
  Orthogonality: $\sum_i W_{ij} \cdot Q_{ij} \leq \beta \quad \forall j$ (smallest feasible β)
  $W_{ij} \in \{0, 1\}$
$\mathbf{D} \leftarrow \{(i, \{j : W_{ij} = 1\}) \mid \forall i\}$

Step 4: Evaluation
Compute NMI, ACC (Hungarian), ARI, Silhouette against ground-truth labels
Compute TC, ITF for each cluster description in $\mathbf{D}$

3.3. Feature Extraction

We use ResNet-101 [He et al., 2016] pretrained on ImageNet [Deng et al., 2009] as our feature extractor. The final classification layer is removed, and the output of the global average pooling layer provides a 2048-dimensional feature vector for each image. All backbone weights are frozen during the entire pipeline—no fine-tuning is performed.

Images are preprocessed with standard ImageNet normalization: resized to 224×224 pixels, converted to tensors, and normalized with mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$. Features are extracted once and cached to disk for reuse across experiments.

This approach follows DDC [Zhang and Davidson, 2021], which uses the same frozen, pretrained ResNet-101 features—DDC additionally trains a small encoder on top of them, whereas we cluster the features directly. The use of pretrained features is well-established in the clustering literature; K-means on these frozen ResNet-101 features already achieves NMI = 0.869 on AwA2, demonstrating that the backbone captures sufficient class-discriminative information without task-specific training.

3.4. Clustering Methods

We evaluate four clustering configurations that systematically vary the clustering method and the presence of interpretable descriptions:

Mode	Clustering	Interpretability
`kmeans`	K-means	—
`deccs`	DECCS consensus	—
`ddc`	K-means + ILP	Cluster descriptions
`ddeccs`	DECCS consensus + ILP	Cluster descriptions

Table 3.1: Four experimental modes. DDECCS (bottom row) is the thesis contribution, combining consensus clustering with interpretable descriptions. The clustering assignments of `ddc` are identical to `kmeans`, and those of `ddeccs` are identical to `deccs`; the modes differ only in whether the post-hoc ILP description step is applied.

A note on the `ddc` and `ddeccs` mode names. The `ddc` mode is *not* a reimplementation of the original Deep Descriptive Clustering method of Zhang and Davidson [Zhang and Davidson, 2021]. The original DDC trains a small encoder (three fully-connected layers) on top of frozen ResNet-101 features, maximizing mutual information and using a self-generated pairwise loss, and solves the ILP *iteratively inside* the training loop so that the explanation can reshape the clustering. In contrast, our `ddc` mode performs no training: it applies the ILP *once, post-hoc*, to the K-means clustering of frozen features. We use the name only because the mode pairs a single-algorithm clustering with ILP-based descriptions, mirroring DDC's combination at a high level. The `deccs` and `ddeccs` modes are named analogously: as detailed in Section 3.4.2, our consensus step adapts the consensus idea of DECCS in a post-hoc fashion rather than reimplementing its trained representation learning. All comparisons to the original DDC in Chapter 4 use its published results; Chapter 5 reports a separate reproduction of the original method, built from its published specification, which supplies the controlled comparison those published figures cannot.

3.4.1 K-means Baseline

K-means clustering [Lloyd, 1982] is applied directly to the 2048-dimensional ResNet features with K clusters and 10 random initializations, using scikit-learn's default Lloyd solver. This serves as the baseline against which consensus clustering is compared.

3.4.2 DECCS Consensus Clustering

DECCS [Miklautz et al., 2022] improves clustering robustness through ensemble consensus. Our implementation builds a consensus from four base clustering algorithms applied to the standardized features. Each algorithm makes a different assumption about cluster shape—its *inductive bias*—and these assumptions are complementary, which is precisely what makes the consensus robust: a grouping that several algorithms with different biases agree on is unlikely to be an artifact of any single one.

1. **K-means** assigns each point to the nearest of K centroids and minimizes within-cluster variance. It implicitly assumes clusters are convex, roughly spherical, and of similar size, and is fast but fails on elongated or non-convex clusters. The ensemble member uses the Elkan algorithm for efficiency; the standalone `kmeans` baseline uses scikit-learn's default.

2. **Spectral Clustering** builds a k-nearest-neighbour affinity graph ($k = 15$) and clusters using the eigenvectors of the graph Laplacian. Because it operates on graph connectivity rather than raw distances, it recovers non-convex, manifold-shaped clusters that K-means cannot.

3. **Gaussian Mixture Model (GMM)** models the data as a mixture of K Gaussians with full covariance, giving each point a soft probabilistic membership and capturing elliptical clusters of differing orientation and size. Estimating a full covariance matrix in 2048 dimensions is numerically unstable, so we first reduce the features with PCA before fitting the GMM.

4. **Agglomerative Clustering** starts with each point as its own cluster and repeatedly merges the pair whose merger least increases within-cluster variance (Ward linkage). It is hierarchical and makes no centroid or distributional assumption, capturing nested structure.

The ensemble results are combined via a sparse co-association matrix A, where A_{ij} counts how often instances i and j are assigned to the same cluster across the base clusterers, restricted to each instance's $k = 20$ nearest neighbours for efficiency [Fred and Jain, 2005]. Writing B_{ij} for the number of ensemble members that place i and j together, restricted to neighbour pairs, the matrix is symmetrized as $A \leftarrow \frac{1}{4}(B + B^\top)$, so a pair that is mutually within each other's neighbourhood carries twice the weight of a one-directional pair and entries range over $[0, 2]$ rather than $[0, 1]$. Because the final step treats A as an affinity graph, only the relative edge weights matter. The final clustering applies Spectral Clustering to this consensus matrix.

We apply Spectral Clustering rather than K-means to this matrix because the co-association values define a graph—instances are nodes and co-association frequencies are edge weights—and Spectral Clustering is designed to partition exactly this kind of

graph structure; running K-means on the matrix would instead treat the co-association values as raw features and discard the graph interpretation [Fred and Jain, 2005].

This differs from the original DECCS, which trains an autoencoder with a consensus loss during representation learning. Our post-hoc consensus applies the ensemble after feature extraction, trading the iterative refinement of learned representations for the reliability and reproducibility of frozen pretrained features. As our experiments demonstrate (Section 4.4), this trade-off is favorable: DECCS consensus provides significant improvements on datasets where individual clustering methods are less reliable (e.g., +15.6% relative NMI on aPY's 32-class task).

3.5. Cluster Description via Integer Linear Programming

Following DDC [Zhang and Davidson, 2021], we generate interpretable cluster descriptions by solving an Integer Linear Programming (ILP) problem that selects a small, orthogonal set of semantic attributes for each cluster.

3.5.1 Problem Formulation

Given cluster assignments $\{C_1, \ldots, C_K\}$ and a set of M semantic attributes, we define the attribute coverage matrix $Q \in \mathbb{R}^{K \times M}$ where Q_{ij} is the mean value of attribute j across all instances in cluster C_i. The ILP solves for a binary allocation matrix $W \in \{0,1\}^{K \times M}$ where $W_{ij} = 1$ indicates that cluster C_i is described by attribute j.

Objective (DDC Eq. 2): Minimize the total description length summed over clusters:

$$\arg\min_{W} \sum_{i,j} W_{ij} \tag{3.1}$$

Coverage constraint (DDC Eq. 3): Each cluster must have sufficient attribute coverage:

$$\sum_{j=1}^{M} W_{ij} \cdot Q_{ij} \geq \alpha \quad \forall i \in \{1, \ldots, K\} \tag{3.2}$$

Orthogonality constraint (DDC Eq. 4): the total coverage-weighted usage of each attribute across all clusters is bounded by β, which keeps descriptions distinctive by discouraging any single attribute from being shared widely across clusters:

$$\sum_{i=1}^{K} W_{ij} \cdot Q_{ij} \leq \beta \quad \forall j \in \{1, \ldots, M\} \tag{3.3}$$

The solver searches for the smallest feasible β starting from $\beta = 1$, ensuring the tightest possible orthogonality. We use the PuLP library [Mitchell et al., 2011] with the CBC solver and a 30-second timeout per β value, after which that β is treated as infeasible and the search continues.

3.5.2 Adaptive α Parameter

DDC uses a fixed $\alpha = 8$, which assumes dense attribute annotations. For datasets with sparse attributes, $\alpha = 8$ can make the ILP infeasible, and even when it is feasible it forces overly long descriptions. We therefore introduce an adaptive α that scales with the mean attribute value, and explain the issue with a concrete example.

The coverage constraint (Eq. 3.2) requires $\sum_j W_{ij} \cdot Q_{ij} \geq \alpha$ for each cluster, where the Q_{ij} are the mean attribute values within the cluster. A cluster's maximum achievable coverage is therefore $\sum_j Q_{ij}$, which is small when the attributes are sparse.

On aPY the attribute matrix is sparse, with mean value $\bar{d} \approx 0.087$ on the 15-class subset. Its entries are per-class means of the binary per-instance annotations (Section 4.2.2), so a typical cluster has Q values like Wheel = 0.30, Metal = 0.25, Pedal = 0.15, Furry = 0.02—small because each attribute holds for only a small fraction of a class's instances. Even selecting all 64 attributes yields a coverage of only about $64 \times 0.087 \approx 5.6$ for a typical cluster—well below 8—so $\alpha = 8$ is infeasible on aPY. The paper's own analysis of α independently observes that smaller values yield more concise and more orthogonal explanations [Zhang and Davidson, 2021], so scaling α down on sparse data costs nothing in description quality.

On AwA2, attributes are continuous (normalized to $[0, 1]$) with mean value $\bar{d} \approx 0.217$. The dominant attributes in a clean cluster have high coverage—for a tiger cluster, *stripes*, *orange*, and *muscle* are each around 0.85–0.9—so four such attributes already sum to roughly 3.4. Here $\alpha = 8$ would be feasible but would force each cluster to include ten or more attributes to accumulate that much coverage, producing verbose, less distinctive descriptions.

Our adaptive formula scales α proportionally to $\bar{d}$:

$$\alpha = \text{clip}\left(\text{round}(16 \cdot \bar{d}),\ 2,\ 8\right) \tag{3.4}$$

where $\bar{d}$ is the mean attribute value across all instances and all attributes (i.e., the average entry in the attribute matrix, min–max normalized to $[0, 1]$); were the attributes strictly binary this would reduce to the fraction of non-zero entries. The constant 16 recovers DDC's $\alpha = 8$ at a reference density of $\bar{d} = 0.5$ and scales the requirement down for sparser attributes. The clip to $[2, 8]$ guarantees at least two meaningful attributes per cluster (lower bound) and never exceeds DDC's original setting (upper bound). This yields $\alpha = 3$ for AwA2 ($\bar{d} = 0.217$) and $\alpha = 2$ for aPY (both the 15-class subset, $\bar{d} = 0.087$, and the full 32-class set, $\bar{d} = 0.111$), producing concise descriptions of roughly four to six attributes per cluster on both datasets.

3.6. Design Rationale: Why Frozen Features Instead of a Trained Representation

Our initial approach attempted to replicate DDC's trained-encoder approach—a trainable MLP on frozen ResNet features, optimized with mutual information maximization and self-generated pairwise constraints. This did not scale: the trained representation collapsed on the full datasets, performing far worse than K-means on the same frozen features. A later code review additionally found implementation defects in these variants, so the collapse reflects our implementations rather than a verdict on trained approaches as such. We give the full diagnosis in Sections 4.7 and 6.2.

This outcome motivates the design above. Rather than training features that already separate the classes well, we focus the contribution on two decoupled components: consensus clustering for robustness, and post-hoc ILP descriptions for interpretability. Decoupling them lets us evaluate clustering quality and description quality independently and yields a deterministic, reproducible pipeline free of training instabilities—at the cost of task-specific feature adaptation, a trade-off we examine in Chapter 6.

4. Experiments

This chapter presents the experimental evaluation of DDECCS. We first describe the experimental setup and implementation (Section 4.1), the datasets (Section 4.2), and the evaluation and interpretability metrics (Section 4.3). We then report the main clustering results across all modes and datasets and compare them against DDC's published numbers (Section 4.4), present the generated cluster descriptions (Section 4.5), and visualize the clusterings (Section 4.6). Finally, we document the early experiments that informed the final design (Section 4.7).

4.1. Experimental Setup

All experiments were conducted using the pipeline described in Chapter 3. We evaluate four modes—`kmeans`, `deccs`, `ddc`, and `ddeccs`—on two datasets: AwA2 (50 classes, 85 attributes, 29857 training instances) and aPY (evaluated with both 32 classes and the DDC-matching 15-class subset). Following DDC's protocol, each experiment is repeated 10 times with different random seeds, and we report mean ± standard deviation.

Experiments were run on an NVIDIA GeForce RTX 4090 GPU. Feature extraction takes approximately 90 seconds per dataset; subsequent clustering and evaluation take seconds for K-means and 5–20 minutes for DECCS consensus (depending on dataset size). Features are cached to disk after the first extraction, so repeated runs skip this step entirely.

4.1.1 Implementation Details

The implementation uses PyTorch for feature extraction, scikit-learn for clustering algorithms, and PuLP with the CBC solver for the ILP. Key parameters:

Table 4.1: Configuration parameters for all experiments.

Parameter	**Value**
Feature extractor	ResNet-101, ImageNet pretrained, frozen
Feature dimension	2048
Image resolution	224×224
Train/test split	80/20 (seed = 42)
K-means initializations	10
DECCS base clusterers	KMeans, Spectral, GMM, Agglomerative
GMM PCA reduction	$\min(256, \dim, \max(64, N/2K))$ dims (for numerical stability)
ILP solver	PuLP CBC, 30s timeout per β
ILP α	3 (AwA2), 2 (aPY) — adaptive, see Eq. 3.4
Number of runs	10 (different seeds)

4.2. Datasets

We evaluate on two established attribute-annotated image datasets used in the DDC literature.

4.2.1 Animals with Attributes 2 (AwA2)

AwA2 [Xian et al., 2019] contains 37,322 images across 50 animal classes, each annotated with 85 continuous semantic attributes (e.g., *furry*, *stripes*, *swims*). Attributes are defined per class—all instances of a given class share the same attribute vector. We use the standard 80/20 train/test split, yielding 29857 training instances.

We note that DDC's original evaluation uses the AwA1 dataset [Lampert et al., 2014] (19832 images), which has been withdrawn due to copyright restrictions. AwA2 is the recommended drop-in replacement, sharing the same class definitions and attribute structure but containing different images. This means direct numerical comparison with DDC's reported results on AwA1 is approximate.

4.2.2 Attribute Pascal and Yahoo (aPY)

aPY [Farhadi et al., 2009] contains 15339 object instances across 32 classes from Pascal VOC 2008 and Yahoo images, annotated with 64 binary semantic attributes (e.g., *Wheel*, *Furry*, *Metal*). In the raw annotations, attributes in aPY are recorded per instance—each object has its own binary attribute vector based on its bounding box crop. Our preprocessing averages these per-instance annotations into a per-class mean attribute vector, so that both datasets enter the pipeline in the same form: one continuous attribute vector per class. This discards within-class attribute variation, and Section 7.3 returns to it as a limitation.

DDC reports results on a 15-class subset of aPY consisting of 5,274 instances. In our dataset, filtering to these 15 classes yields 6,487 instances; after our standard 80/20 train/test split, we cluster on 5,189 training instances. The discrepancy with DDC's reported count likely reflects differences in preprocessing or filtering criteria. Following their setup, we evaluate with both the full 32-class dataset and the 15-class subset matching DDC's configuration. The 15 classes, identified from DDC's Figure 3 ontology, are: dog, cat, monkey, zebra, bird, bag, mug, bottle, chair, car, building, bicycle, boat, sofa, and dining table.

4.3. Evaluation and Interpretability Metrics

We assess results along two axes: how well clusters recover the ground-truth classes, and how interpretable the generated descriptions are. The former is captured by four standard clustering metrics (Section 4.3.1); the latter by the Tag Coverage and Inverse Tag Frequency metrics introduced by DDC (Section 4.3.2).

4.3.1 Clustering Metrics

We evaluate clustering quality using four standard metrics:

Normalized Mutual Information (NMI) [Strehl and Ghosh, 2002] measures the agreement between predicted clusters and ground-truth classes, normalized to $[0, 1]$. NMI $= 1$ indicates perfect agreement.

Clustering Accuracy (ACC) uses the Hungarian algorithm [Kuhn, 1955] to find the optimal one-to-one mapping between predicted clusters and true classes, then computes the fraction of correctly assigned instances.

Adjusted Rand Index (ARI) [Hubert and Arabie, 1985] measures pairwise agreement between clusterings, adjusted for chance. ARI $= 0$ indicates random clustering; ARI $= 1$ indicates perfect agreement.

Silhouette Score [Rousseeuw, 1987] measures how similar instances are to their own cluster compared to other clusters, ranging from -1 to $+1$. Unlike the other three, it requires no ground-truth labels. Because it is quadratic in the number of instances, we compute it on a random subsample of at most 10,000 instances per run; this affects AwA2 and the 32-class aPY set.

4.3.2 Interpretability Metrics

Following DDC, we evaluate cluster descriptions using Tag Coverage (TC) and Inverse Tag Frequency (ITF). For a cluster C_i with description D_i:

Tag Coverage (DDC Eq. 8) averages, over the attributes in a description, how strongly each is present across the cluster's members. DDC states it for binary tags, as the fraction of members possessing the attribute:

$$TC(C_i) = \frac{1}{|D_i|} \sum_{d \in D_i} \frac{|\{(x,t) \in C_i : d \in t\}|}{|C_i|} \tag{4.1}$$

Inverse Tag Frequency (DDC Eq. 9) measures how unique each cluster's description is:

$$ITF(C_i) = \frac{1}{|D_i|} \sum_{d \in D_i} \log \frac{K}{\sum_{j=1}^{K} \mathbb{1}[d \in D_j]} \tag{4.2}$$

where log denotes the natural logarithm.

Because our attribute vectors are continuous rather than binary—AwA2's attributes are continuous by construction, and aPY's per-instance binary annotations are averaged per class (Section 4.2.2)—we evaluate the inner term of Eq. 4.1 as the mean attribute value Q_{ij} over the cluster, which reduces to the fraction of members possessing the attribute in the binary case DDC assumes. TC on our data is therefore a mean attribute intensity rather than a literal membership fraction.

Higher TC indicates that the selected attributes are prevalent within the cluster; higher ITF indicates that the attributes are distinctive to that cluster rather than shared across many clusters.

4.4. Main Results

This section reports clustering performance across the four modes on AwA2, the full aPY dataset, and the 15-class aPY subset, followed by a comparison with DDC's published results. Recall from Chapter 3 that the four modes yield only *two* distinct clusterings: `ddc` reuses the `kmeans` clustering and `ddeccs` reuses the `deccs` clustering, each adding post-hoc ILP descriptions. The clustering metrics (NMI, ACC, ARI, Silhouette) are

therefore identical within each pair; the description modes differ only in their TC and ITF values.

4.4.1 AwA2 Dataset (K = 50)

Table 4.2 presents the clustering results on the full AwA2 dataset with $K = 50$ clusters.

Table 4.2: Clustering performance on AwA2 (29857 samples, 50 classes). Mean ± std over 10 runs; Silhouette is reported as the mean only. Here `ddc` and `ddeccs` are our K-means+ILP and consensus+ILP description modes, *not* the original DDC; their clustering metrics therefore match `kmeans` and `deccs`, and only TC/ITF are added.

Mode	NMI	ACC	ARI	Sil	TC	ITF
`kmeans`	0.869 ± 0.002	0.785 ± 0.008	0.744 ± 0.008	0.189	—	—
`deccs`	0.871 ± 0.001	0.807 ± 0.002	0.738 ± 0.001	0.160	—	—
`ddc`	0.869 ± 0.002	0.785 ± 0.008	0.744 ± 0.008	0.189	0.681	2.652
`ddeccs`	0.871 ± 0.001	0.807 ± 0.002	0.738 ± 0.001	0.160	0.698	2.656

As the caption notes, the `ddc`/`ddeccs` rows add only the TC and ITF columns; their clustering metrics are inherited from `kmeans`/`deccs`. The comparison with the original DDC's published numbers appears in Section 4.4.5.

On AwA2, all four modes achieve strong clustering performance (NMI ≈ 0.87). K-means and DECCS consensus produce nearly identical NMI (0.869 vs. 0.871). DECCS consensus shows notably higher ACC (0.807 vs. 0.785) and markedly lower run-to-run variance, though ARI is marginally lower (0.738 vs. 0.744); on AwA2 the consensus mainly improves accuracy and stability rather than every metric uniformly, producing more reproducible clusters than K-means alone even when NMI is similar. Across the ten seeds the consensus reduces the NMI standard deviation by a factor of roughly three and a half, and the ACC and ARI standard deviations by factors of roughly five and seven respectively.

4.4.2 aPY Dataset — Full (K = 32)

Table 4.3 presents results on the full aPY dataset with all 32 classes.

Table 4.3: Clustering performance on aPY full (12,271 samples, 32 classes). Mean ± std over 10 runs; Silhouette is reported as the mean only. Here `ddc` and `ddeccs` are our K-means+ILP and consensus+ILP description modes, *not* the original DDC; their clustering metrics therefore match `kmeans` and `deccs`, and only TC/ITF are added.

Mode	NMI	ACC	ARI	Sil	TC	ITF
`kmeans`	0.534 ± 0.014	0.390 ± 0.016	0.138 ± 0.012	0.013	—	—
`deccs`	0.618 ± 0.006	0.578 ± 0.034	0.297 ± 0.046	−0.018	—	—
`ddc`	0.534 ± 0.014	0.390 ± 0.016	0.138 ± 0.012	0.013	0.497	2.156
`ddeccs`	0.618 ± 0.006	0.578 ± 0.034	0.297 ± 0.046	−0.018	0.574	2.440

On aPY, DECCS consensus provides a substantial improvement over K-means: NMI increases from 0.534 to 0.618 (+15.6% relative), ACC from 0.390 to 0.578 (+48.0%), and ARI from 0.138 to 0.297 (+114.8%). Its effect on variance is metric-dependent:

DECCS reduces the NMI standard deviation by roughly a factor of 2.5 (±0.014 to ±0.006) but increases it for ACC (±0.016 to ±0.034) and ARI (±0.012 to ±0.046).

4.4.3 aPY Dataset — DDC 15-Class Subset (K = 15)

To enable direct comparison with DDC's published results, we evaluate on the same 15-class subset used in the DDC paper.

Table 4.4: Clustering performance on aPY 15-class subset (5189 training samples, 15 classes). Mean ± std over 10 runs; Silhouette is reported as the mean only. Here `ddc` and `ddeccs` are our K-means+ILP and consensus+ILP description modes, *not* the original DDC; their clustering metrics therefore match `kmeans` and `deccs`, and only TC/ITF are added.

Mode	NMI	ACC	ARI	Sil	TC	ITF
`kmeans`	0.666 ± 0.028	0.615 ± 0.046	0.458 ± 0.048	0.027	—	—
`deccs`	0.684 ± 0.005	0.658 ± 0.012	0.477 ± 0.014	0.032	—	—
`ddc`	0.666 ± 0.028	0.615 ± 0.046	0.458 ± 0.048	0.027	0.447	2.050
`ddeccs`	0.684 ± 0.005	0.658 ± 0.012	0.477 ± 0.014	0.032	0.493	2.228

On the 15-class subset, results are closer between K-means and DECCS. DECCS achieves slightly higher NMI (0.684 vs. 0.666) and higher ACC (0.658 vs. 0.615), and reduces run-to-run variance on every metric—by roughly a factor of six on NMI (±0.028 to ±0.005) and close to four on ACC and ARI.

4.4.4 Feature Standardization Ablation

The K-means baseline clusters the raw 2048-dimensional ResNet features, whereas the DECCS ensemble standardizes them to zero mean and unit variance before clustering (Section 3.4.2). Because the two arms of our main comparison therefore differ in preprocessing as well as in algorithm, we verify that the reported consensus gains are not an artifact of standardization by rerunning the K-means baseline on standardized features under identical seeds.

Table 4.5: K-means on raw versus standardized features, mean ± std over 10 runs. Standardization reduces NMI on all three configurations and drives the Silhouette score negative.

	NMI		Silhouette	
Dataset	raw	standardized	raw	standardized
AwA2	0.869 ± 0.002	0.777 ± 0.017	0.189	−0.057
aPY (32-class)	0.534 ± 0.014	0.520 ± 0.014	0.013	−0.031
aPY (15-class)	0.666 ± 0.028	0.573 ± 0.024	0.027	−0.026

Standardization does not help K-means on these features; it costs 0.093 NMI on AwA2, 0.093 on aPY-15, and 0.014 on aPY-32, and turns the Silhouette score negative in every case, meaning the average instance sits closer to a neighbouring cluster than to its own. Per-dimension scale in ResNet-101 activations evidently carries class information that whitening discards. The consensus gains reported above are therefore

not attributable to preprocessing: the ensemble reaches NMI = 0.618 on aPY-32 from a representation on which K-means manages only 0.520. Because raw features give the stronger single-algorithm baseline, we retain them for the comparison in Tables 4.2–4.4, which makes the reported improvements conservative.

4.4.5 Comparison with DDC Paper

Table 4.6 compares our results with those reported in the DDC paper [Zhang and Davidson, 2021]. We stress that the rows labelled "DDC paper" are the original DDC method (a trained encoder with MI and pairwise losses), whereas our rows use frozen ResNet features with K-means/DECCS and post-hoc ILP; the `ddc` mode of the previous tables is not shown here, as its clustering is identical to our K-means.

Table 4.6: Comparison with DDC paper results. DDC uses AwA1 (not AwA2) and trains an MLP with MI + pairwise loss. Our approach uses frozen ResNet features + K-means/DECCS + post-hoc ILP. DDC results at tag ratio $r\% = 50$.

Dataset	Method	NMI	ACC
AwA (K=40 vs. K=50)	DDC paper: K-means	74.23 ± 0.69	68.24 ± 0.54
	DDC paper: DDC	89.57 ± 1.37	84.48 ± 1.20
	Ours: K-means	86.94 ± 0.17	78.52 ± 0.80
	Ours: DDECCS	87.14 ± 0.05	80.74 ± 0.17
aPY (K=15)	DDC paper: K-means	64.38 ± 0.48	58.98 ± 0.37
	DDC paper: DDC	86.30 ± 1.22	79.87 ± 1.02
	Ours: K-means	66.55 ± 2.80	61.53 ± 4.58
	Ours: DDECCS	68.41 ± 0.50	65.77 ± 1.20

A global caveat. No row in Table 4.6 compares methods on identical data. The DDC figures here are published values, and the `ddc` and `ddeccs` modes reuse only its ILP description module; its training objective was attempted during development and did not scale to our datasets (Section 4.7). A controlled comparison against a reproduction of the full original method, run on our own features and splits, is reported separately in Chapter 5. On AwA, DDC uses the withdrawn AwA1 (19,832 instances, $K = 40$) whereas we use AwA2 (29,857 instances, $K = 50$). On aPY, the same 15 classes yield 6,487 instances in our data versus DDC's reported 5,274 (about 23% more); we cluster on our 5,189-instance training split while DDC clusters on its full set, so the underlying populations differ even though the per-set sizes are similar. Every DDC figure is taken from the published paper, not from a controlled re-run on our data and splits. The comparison should therefore be read as indicative of relative standing rather than exact head-to-head measurements. We interpret these differences in Section 6.1.3.

Note on DDC's tag annotation ratio ($r\%$). DDC's results in Table 4.6 are reported at tag annotation ratio $r = 50\%$, meaning half of each instance's attributes are randomly masked during training. This masking is part of DDC's *training procedure*—it simulates incomplete annotation and creates diversity in pairwise constraint generation. At lower annotation ratios—i.e. more tags withheld—DDC reports lower performance (NMI = 75.62 on AwA at $r = 10\%$). DDC does not report results at $r = 0\%$, so we cannot determine how much the masking itself contributes. Our pipeline does not

use masking, since we do not train with pairwise constraints; we use full (unmasked) attributes only for the post-hoc ILP description step, which computes cluster-level attribute statistics from the attribute matrix. The original DDC's ILP therefore sees per-instance binary tags at $r = 0.5$, whereas ours sees unmasked per-class means on both datasets—a methodological difference that may contribute to the description gap on aPY.

4.5. ILP Cluster Descriptions

A key contribution of our approach is the generation of interpretable cluster descriptions. We focus on the ILP-optimized descriptions, which select attributes that are both prevalent within a cluster (coverage constraint) and distinctive to it (orthogonality constraint). This contrasts with a naive top-5 ranking by mean attribute value, which would surface dataset-wide attributes such as "Occluded" that appear in many clusters; the orthogonality constraint instead bounds how widely such attributes can be reused—"Occluded" is retained in only four of the 32 aPY clusters—and favors ones like "Pedal" and "Handlebars" that are specific to a single cluster on the 15-class subset.

These are our modes, not the DDC paper. The descriptions in this section are generated by our pipeline (the `ddc` and `ddeccs` modes), not taken from the original DDC paper. The original DDC publishes per-cluster TC/ITF only for a controlled 10-class AwA subset (where it reports TC = 1.00 for every cluster), and does not report per-dataset average TC/ITF for its full AwA or aPY runs. Section 4.5.2 reports the limited comparison that DDC's published figures allow. Chapter 5 reports description metrics for a reproduction of the original method, though not on a directly comparable footing: it uses each dataset's binary attribute matrix rather than the continuous one used here, so its TC and ITF values are computed over a different Q. We can more directly compare our two description modes, which differ only in the clustering they describe.

4.5.1 Description Quality: `ddc` vs. `ddeccs`

Table 4.7 compares the average TC and ITF of our two description modes across all three datasets. The `ddeccs` mode (ILP applied to the consensus clustering) achieves higher TC and ITF than `ddc` (ILP applied to the K-means clustering) on every dataset. This indicates that the more coherent clusters produced by consensus yield descriptions that are both more representative of their members and more distinctive across clusters.

Table 4.7: Average TC and ITF for our two ILP description modes. `ddc` applies the ILP to the K-means clustering; `ddeccs` applies it to the DECCS consensus clustering.

	`ddc`		`ddeccs`	
Dataset	**TC**	**ITF**	**TC**	**ITF**
AwA2	0.681	2.652	0.698	2.656
aPY (32-class)	0.497	2.156	0.574	2.440
aPY (15-class)	0.447	2.050	0.493	2.228

4.5.2 Reported Description Metrics of the Original DDC

The original DDC reports average TC and ITF only for a down-sampled AwA setting (10 classes clustered into 5). Table 4.8 places those figures alongside our `ddeccs` descriptions on AwA2. The two are not directly comparable; Section 6.1 explains why.

Table 4.8: Average TC and ITF reported by the original DDC versus our DDECCS on AwA. The two are not directly comparable; see Section 6.1.

Method	K	α	Avg TC	Avg ITF
Original DDC (AwA, 10 cls $\rightarrow$ 5 clusters)	5	8	1.00	2.32*
DDECCS, `ddeccs` mode (AwA2, 50 classes)	50	3	0.698	2.656

*DDC computes ITF with $\log_2$, whereas we use the natural logarithm. On the natural-log scale DDC's value is $2.32 \ln 2 \approx 1.61 = \ln 5$ — the maximum possible for $K = 5$, i.e. every descriptive tag is unique to one cluster.

4.5.3 AwA2 ILP Descriptions

Table 4.9 shows selected `ddc`-mode descriptions for AwA2 clusters. The ILP solved with $\alpha = 3$, $\beta = 3$, selecting 73 of 85 attributes with an average of 4.7 attributes per cluster.

Table 4.9: Selected ILP cluster descriptions for AwA2 (our `ddc` mode). Full descriptions available in `results/awa2/ddc/ilp_descriptions.json`.

C	Description	Dominant Class	TC	ITF
0	strong, fish, arctic, fierce	polar bear	0.76	2.77
3	stripes, chewteeth, plains, group	zebra	0.76	2.61
13	orange, stripes, muscle, fierce	tiger	0.82	3.12
17	bush, jungle, tree, smart	chimpanzee	0.78	3.19
34	gray, hairless, tusks, oldworld	elephant	0.76	2.94
35	spots, longleg, longneck, plains	giraffe	0.77	2.89

Table 4.10: Selected ILP cluster descriptions for AwA2 (our `ddeccs` mode, the proposed method). Dominant classes confirmed from per-cluster sample images. Full descriptions in `results/awa2/ddeccs/ilp_descriptions.json`.

C	Description	Dominant Class	TC	ITF
17	orange, stripes, claws, muscle	tiger	0.80	3.09
26	strong, fish, arctic, fierce	polar bear	0.76	2.77
21	hands, bipedal, jungle, group	chimpanzee	0.77	2.87
33	gray, toughskin, tusks, oldworld	elephant	0.78	2.70
4	spots, longleg, longneck, grazer	giraffe	0.77	2.89

Table 4.9 lists representative `ddc`-mode clusters with their descriptions and TC/ITF values—for example, cluster 13 (tiger) at TC = 0.82, ITF = 3.12, and cluster 17 (chimpanzee), described by "bush, jungle, tree, smart." Table 4.10 shows the corresponding `ddeccs`-mode descriptions for the same classes; the TC/ITF values are near-identical (the tiger cluster at 0.82/3.12 versus 0.80/3.09) even where the selected attributes differ, confirming that consensus preserves description quality. We interpret these descriptions in Section 6.1.

4.5.4 aPY Cluster Descriptions (15-class subset)

Table 4.11 shows `ddc`-mode descriptions for the aPY 15-class subset. The ILP solved with $\alpha = 2$, $\beta = 1$, selecting 43 of 64 attributes with 6.0 attributes per cluster. The low orthogonality bound ($\beta = 1$) keeps each attribute's coverage-weighted usage across clusters small, so the descriptions stay distinctive.

Table 4.11: Selected ILP cluster descriptions for aPY 15-class subset (our `ddc` mode). Full descriptions in `results/apy_15/ddc/ilp_descriptions.json`.

C	Description	Dominant Class	TC	ITF
1	Wheel, Door, Taillight, Side mirror, Exhaust	car	0.41	2.07
4	Furn. Back, Furn. Seat, Furn. Arm, Leather	chair/sofa	0.51	2.01
9	Leg, Foot/Shoe, Furry	mixed animals	0.70	2.48
12	Tail, Ear, Snout	dog/cat	0.69	2.71
14	Pedal, Handlebars, Text, Plastic	bicycle	0.53	2.01

Table 4.12: Selected ILP cluster descriptions for aPY 15-class subset (our `ddeccs` mode, the proposed method). Dominant classes are confirmed from per-cluster sample images. Full descriptions in `results/apy_15/ddeccs/ilp_descriptions.json`.

C	Description	Dominant Class	TC	ITF
6	Wheel, Door, Headlight, Side mirror, Exhaust	car	0.41	2.15
5	Pedal, Handlebars, Text, Plastic	bicycle	0.52	2.01
7	Tail, Snout, Furry	cat	0.70	2.71
14	Beak, Foot/Shoe, Feather	bird	0.71	2.48
1	Vert Cyl, Label, Clear, Shiny	bottle	0.52	2.01
12	2D Boxy, Round, Plastic, Glass, Clear, Shiny	mug	0.34	1.95

Table 4.11 lists representative aPY clusters with their descriptions and TC/ITF values; the car (C1) and bicycle (C14) clusters receive disjoint attribute sets, and cluster 12 reaches the maximum possible ITF for $K = 15$ ($\ln 15 \approx 2.71$). Section 6.1 discusses what these descriptions reveal.

Interestingly, the representative car attribute differs between modes—"Taillight" in `ddc` (Table 4.11) and "Headlight" in `ddeccs` (Table 4.12)—because the consensus groups the car instances slightly differently, illustrating that the descriptions track the underlying clustering rather than being fixed to a class.

4.5.5 aPY Cluster Descriptions (Full 32-class)

Table 4.13 shows verified `ddeccs`-mode descriptions on the full 32-class aPY dataset (we show the proposed mode here; per-cluster `ddc` and `ddeccs` listings for all clusters are in Appendix B). The most distinctive cluster, potted plant, reaches the maximum ITF of $\ln 32 \approx 3.47$.

Table 4.13: Selected ILP cluster descriptions for the full aPY dataset (our `ddeccs` mode). Dominant classes confirmed from per-cluster sample images.

C	Description	Dominant Class	TC	ITF
4	Leaf, Pot, Vegetation	potted plant	0.85	3.47
10	Tail, Beak, Feather	bird	0.77	3.00
12	Horiz Cyl, Wing, Propeller, Jet engine	aeroplane	0.55	3.12
24	Wheel, Door, Headlight, Shiny	car	0.51	2.10
21	Vert Cyl, Glass, Shiny	mug	0.71	2.27
9	Vert Cyl, Label, Glass	bottle	0.67	2.73

4.6. Visualizations

Because the ILP description module operates post-hoc, the four modes produce only two distinct clusterings per dataset: `ddc` matches `kmeans`, and `ddeccs` matches `deccs`. We therefore visualize the two genuinely distinct clusterings—K-means versus DECCS consensus—which illustrate how ensemble consensus changes cluster boundaries. On AwA2 the differences are subtle, since both achieve similar NMI; on aPY they are more pronounced, reflecting DECCS's larger improvement.

4.6.1 AwA2: K-means vs. DECCS Consensus

Figure 4.1 projects the AwA2 features with t-SNE under both clusterings.

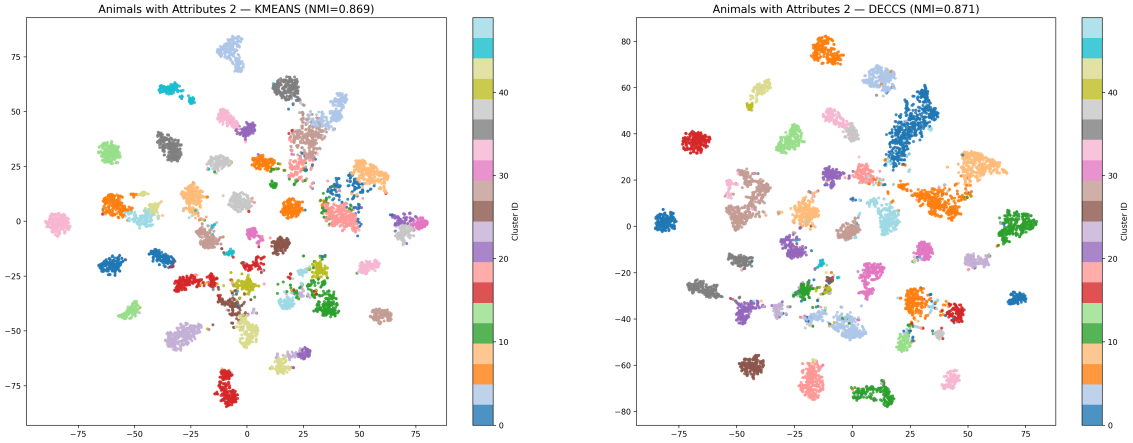

Figure 4.1: t-SNE projections of AwA2 ResNet features colored by cluster assignment. Left: K-means (NMI = 0.869). Right: DECCS consensus (NMI = 0.871). Both produce well-separated clusters, reflecting the strong discriminative power of ResNet features on animal images. (The projection shows a single representative seed; the tables report 10-run means.) Source: `results/awa2/kmeans/tsne.png` and `results/awa2/deccs/tsne.png`.

4.6.2 aPY Full (K = 32): K-means vs. DECCS Consensus

Figure 4.2 shows the same projection for the full 32-class aPY dataset, where the consensus gain is largest.

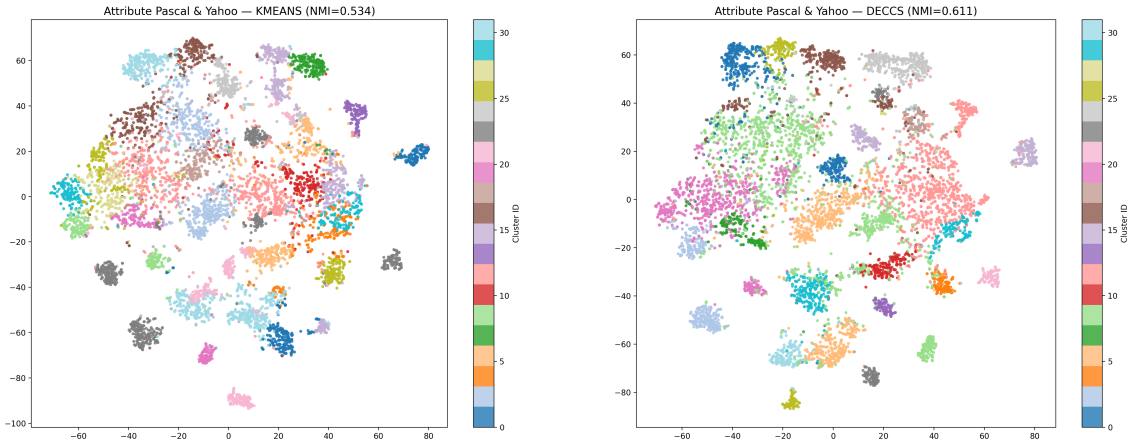

Figure 4.2: t-SNE projections of aPY full dataset (32 classes). Left: K-means (NMI = 0.534). Right: DECCS consensus (NMI = 0.611). The values in the panel titles are for the plotted seed; the 10-run means are 0.534 and 0.618 (Table 4.3). The consensus ensemble produces clusters that align more closely with the true object classes, particularly in separating vehicles from furniture and small objects—the largest gain from consensus in our experiments. As discussed in Section 6.1, this class-level improvement coexists with a lower Silhouette score, which measures Euclidean compactness rather than class agreement. (The projection shows a single representative seed; the tables report 10-run means.) Source: `results/apy/kmeans/tsne.png` and `results/apy/deccs/tsne.png`.

4.6.3 aPY 15-Class Subset (K = 15): K-means vs. DECCS Consensus

Figure 4.3 shows the 15-class subset, where the two clusterings are closest.

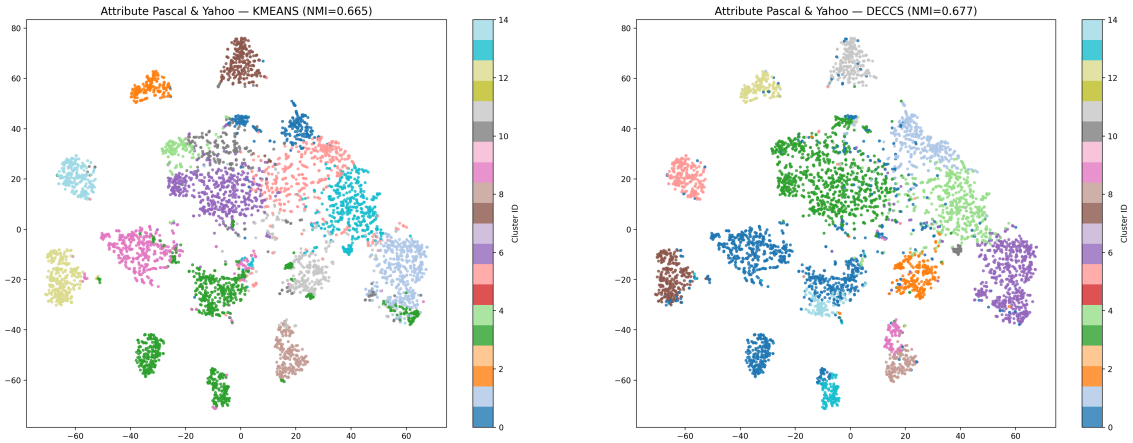

Figure 4.3: t-SNE projections of aPY 15-class subset. Left: K-means (NMI = 0.665). Right: DECCS consensus (NMI = 0.677). The values in the panel titles are for the plotted seed; the 10-run means are 0.666 and 0.684 (Table 4.4). Distinct groups are visible for animals (top), vehicles (left), and furniture/objects (bottom). (The projection shows a single representative seed; the tables report 10-run means.) Source: `results/apy_15/kmeans/tsne.png` and `results/apy_15/deccs/tsne.png`.

4.7. The Path to the Current Approach: Early Experiments

Before arriving at the frozen-features pipeline we tried two trained approaches: a convolutional autoencoder with a tag-prediction branch, and a DDC-style MLP on frozen features trained with mutual-information and self-generated pairwise losses. Both collapsed at full scale—every trained variant fell far below the frozen-features K-means baseline of NMI = 0.869 on AwA2, with the MLP locking into roughly 24 coarse clusters within the first epoch. This is the observation that redirected the work toward clustering the frozen features directly. The full results, and a later code-level diagnosis of why these runs failed, are given in Appendix A.2.

5. A Controlled Comparison with the Original DDC

Chapter 4 compared DDECCS against the published figures of the original DDC, under a global caveat that no row of that comparison placed the two methods on identical data. This chapter removes the caveat. We rebuild the original DDC from its published specification, run it on both datasets under the same features, splits and preprocessing as DDECCS, and report what the controlled comparison shows. We first state why the comparison in Chapter 4 could not be controlled and what was built (Section 5.1), then describe the reproduction and the judgment calls it required (Section 5.2). We then report the matched-baseline results (Section 5.3), isolate which component of the method carries its performance (Section 5.4), measure the description-to-clustering channel directly (Section 5.5), and examine the feasibility of the published configuration on aPY (Section 5.6). Section 5.7 draws the consequences for DDECCS.

5.1. Motivation and Scope

Every comparison between DDECCS and the original DDC reported so far in this thesis rests on numbers taken from the published paper. Those numbers were obtained on a different image set (the withdrawn AwA1 rather than AwA2), at a different number of clusters, on a differently sized aPY subset, and with an unpublished implementation. Section 4.4.5 states this plainly and asks the reader to treat the comparison as indicative of relative standing rather than as a measurement. That is an honest position, but it is a weak one: it leaves open whether the gap between the two methods reflects a difference in algorithms or a difference in evaluation conditions.

No official implementation of the original DDC is available. The authors' homepage links to a paper PDF that no longer resolves and publishes no code, and the DeepClustering survey that catalogues the method lists it with an empty code column. Closing the gap therefore required rebuilding the method from the paper text. We did so, and ran it on AwA2 and on the 15-class aPY subset using the same frozen ResNet-101 features, the same 80/20 split, and the same preprocessing as the rest of this thesis, alongside k-means baselines re-run inside the same harness. This chapter reports that work.

Its purpose is to place DDECCS against a like-for-like baseline, and the results serve that purpose in two ways. They quantify the gap under controlled conditions, which the published figures could not. They also characterise what produces the original DDC's reported performance, and that characterisation bears directly on the design choices DDECCS made: which parts of the original method are load-bearing, whether its clustering numbers measure what they appear to measure, and whether the description-to-clustering feedback loop that DDECCS deliberately omits would have

been worth building. Section 5.7 returns to each of these.

Two boundaries should be stated at the outset. First, this chapter reports a reproduction from a published specification, not a verified replication: where a measurement here disagrees with a published claim, we report it as a failure to replicate under the conditions stated below, with no reference implementation available against which to locate the difference. Second, this work is distinct from the trained variants attempted earlier in the project and documented in Section 4.7. Those were abandoned before the method was correctly implemented; the mechanisms behind their failure were subsequently identified, and Appendix C records them alongside the defects found in the reproduction itself.

5.2. The Reproduction

The original DDC couples three components: a clustering objective that maximizes mutual information between inputs and cluster assignments, a symbolic objective that solves the ILP of Section 3.5 to describe the current partition, and a self-generated pairwise objective that converts the ILP-filtered tag space into pairwise constraints and feeds them back into the clustering. The reproduction implements all three, with the ILP solved inside the training loop rather than once at the end, so that the description module can reshape the clustering as the paper intends. We first summarize what the paper specifies, then the points at which it does not.

5.2.1 The Specification as Implemented

The encoder is a three-layer fully connected network on frozen 2048-dimensional ResNet-101 features, with a softmax output over K clusters. The clustering loss is the mutual information objective of the paper's Equation (1). The ILP is the formulation already given in Equations 3.1–3.3, solved once per round on the current soft assignment, with the resulting allocation matrix W reduced to a tag mask g that gates which attributes contribute to constraint generation. Pairs are generated by the paper's Equation (5): for each anchor, the partner minimizing a score that combines masked tag distance with cluster disagreement, weighted by $\gamma = 100$, with $l = 1$ partner per anchor. The pairwise loss is applied with weight $\lambda = 1$. Tag masking at ratio $r = 0.5$ simulates incomplete annotation. Attribute vectors are the binary predicate matrices distributed with each dataset.

5.2.2 Judgment Calls and One Deviation

Several settings that materially affect the result are not stated in the paper. We list each with the value chosen and the reason, and record the full justifications in Appendix C.3.

Optimization used Adam [Kingma and Ba, 2015] with default parameters, as the paper states, at a learning rate of 10^{-3}, which is Adam's own default and the only defensible choice absent a published value. Batch size was 256, matching the archived earlier attempt so that the two are comparable. The paper trains "until convergence" without a budget; we fixed one at 10 pretraining epochs followed by 20 rounds of 2 epochs each, applied identically to every configuration. Features entered the network raw, without standardization, consistent with Section 4.4.4. The norm in Equation (5) was taken as L_2 and $f_\theta(x)$ as the softmax output. The annotation ratio r was read as

a ratio of annotated *instances*, with the mask drawn once and frozen, rather than of individual entries; pair partners were correspondingly restricted to annotated instances, since under instance-level masking every unannotated instance carries the same imputed vector and would otherwise be selected indiscriminately. Inverse Tag Frequency is reported in nats throughout this thesis, whereas the paper's table is base-2.

One choice is a deviation rather than an interpretation, and it is the most consequential number in this chapter. The published Equation (1) carries no coefficient on the marginal entropy term $H(Y)$. Implemented as written, the objective leaves a large fraction of the clusters unpopulated. We therefore introduce a weight μ on that term and report both settings throughout: $\mu = 1.0$ is the paper-faithful configuration and is the headline reproduction result, while $\mu = 1.5$ is our deviation. Neither is presented without the other.

A further difference concerns the attribute representation and matters for reading the description metrics. On AwA2 the reproduction uses the binary predicate matrix distributed with the dataset, at a mean density of 0.365, whereas the DDECCS pipeline of Chapter 3 uses the continuous per-class matrix at a density of 0.217; the coverage parameter is $\alpha = 3$ in both cases, but the underlying Q matrices differ. On aPY the difference is sharper: this chapter uses the per-instance annotations recovered in Appendix C.4, while Chapter 4 uses their per-class means, so the two are not the same attributes represented differently but different tags altogether. The mean densities there coincide at 0.087, which makes the difference invisible in that statistic. TC and ITF values in this chapter are therefore *not* directly comparable with those in Chapter 4 on either dataset. We repeat this warning wherever the two sets of figures appear close.

5.3. Matched-Baseline Results

Table 5.1 reports the reproduction against k-means baselines computed on the same features, splits and seeds. The k-means rows vary what the baseline is given: the image features alone, the features concatenated with the masked tag block, the same concatenation with the tag block standardized, and the tag block by itself as a control. The last of these is not a competitive baseline but a diagnostic, and Section 5.4.2 explains what it diagnoses.

Table 5.1: The reproduced original DDC against matched k-means baselines, five seeds (42–46), final-epoch values, mean ± std. All rows use the same frozen ResNet-101 features, the same 80/20 split and the same tag mask at $r = 0.5$. "Active" counts clusters receiving at least one instance under argmax assignment; "same-class" is the fraction of self-generated constraint pairs that join two instances of the same true class.

Configuration	NMI	ACC	ARI	Active	Same-class
AwA2 ($N = 29{,}857$, $K = 50$, $\alpha = 3$)					
k-means, features only	0.8699 ± 0.0021	0.7864 ± 0.0099	0.7469	50/50	—
k-means, features ⊕ raw tags	$\mathbf{0.8804 \pm 0.0028}$	0.8033 ± 0.0081	0.7693	50/50	—
k-means, features ⊕ std. tags	0.7665 ± 0.0079	0.5194	0.3767	50/50	—
k-means, tags only (control)	0.5909 ± 0.0000	0.4976 ± 0.0000	0.0439	—	—
DDC, $\mu = 1.0$ (paper-faithful)	0.8081 ± 0.0237	0.4861 ± 0.0454	0.4480	31.0 ± 3.7	0.9215 ± 0.0021
DDC, $\mu = 1.5$ (deviation)	0.9229 ± 0.0062	0.8306 ± 0.0259	0.8276	49.4 ± 0.5	0.9215 ± 0.0021
DDC, $\mu = 1.5$, $\lambda = 0$	0.7425 ± 0.0186	0.6596 ± 0.0318	0.5639	37.4 ± 1.9	0.9215 ± 0.0021
DDC, $\mu = 1.5$, ILP off	0.9255 ± 0.0060	0.8444 ± 0.0222	0.8413	50.0	0.9215 ± 0.0021
aPY-15 ($N = 5{,}189$, $K = 15$, $\alpha = 1$ unless stated)					
k-means, features only	0.6463 ± 0.0196	0.6012	0.4298	15/15	—
k-means, features ⊕ tags	$\mathbf{0.6767 \pm 0.0182}$	0.6166	0.4700	15/15	—
k-means, tags only (control)	0.3778 ± 0.0026	0.3239	0.0544	15/15	—
DDC, $\mu = 1.0$ (paper-faithful)	0.6337 ± 0.0153	0.5630	0.4731	11.2 ± 1.9	0.5202
DDC, $\mu = 1.5$ (deviation)	0.6368 ± 0.0108	0.5610	0.4785	15.0	0.5554
DDC, $\mu = 1.5$, $\alpha = 2$	0.6735 ± 0.0091	0.5974	0.5082	15.0	0.6219
DDC, $\mu = 1.5$, $\lambda = 0$	0.5318 ± 0.0216	0.4751	0.3215	15.0	0.5458
DDC, $\mu = 1.5$, ILP off	$\mathbf{0.6798 \pm 0.0090}$	0.6251	0.5284	15.0	0.6233

The paper-faithful configuration loses to the matched baseline on both datasets. On AwA2 it reaches NMI = 0.8081 against 0.8804 for k-means on features concatenated with the same masked tags. On aPY-15 it reaches 0.6337 against 0.6767, and falls below even the features-only baseline at 0.6463. The margin the paper reports over its own k-means baseline does not appear here; under matched conditions the sign is reversed.

The picture changes with $\mu = 1.5$, but only on AwA2, where the method reaches 0.9229 and does beat the strongest baseline. That configuration is our deviation, not the published method, and the mechanism behind its advantage is the subject of Section 5.4. On aPY-15 the deviation buys almost nothing: 0.6368 against 0.6337, still below both baselines. Raising the coverage parameter to $\alpha = 2$ lifts it to 0.6735, which is within noise of the features-plus-tags baseline and still below the same configuration with the ILP disabled.

The behaviour of the paper-faithful configuration also shows up in the active-cluster count. At $\mu = 1.0$ only 31.0 ± 3.7 of the 50 AwA2 clusters and 11.2 ± 1.9 of the 15 aPY clusters receive any instances, which is why ACC collapses to 0.4861 on AwA2 while NMI remains moderate: NMI tolerates a partition that merges classes, whereas the Hungarian matching underlying ACC does not. The weight on $H(Y)$ is what populates the remaining clusters, and its absence from the published objective is the single clearest gap between the specification and a configuration that runs well.

One comparison in this table deserves a note. The aPY-15 features-only baseline here is 0.6463 ± 0.0196, whereas Chapter 4 reports k-means at 0.666 ± 0.028 for what is nominally the same quantity. The two intervals overlap and the values are consistent, but they are different runs: the figures in this chapter come from five seeds inside the reproduction harness, chosen so that every row of Table 5.1 shares an identical data path, while Chapter 4 reports ten seeds of the DDECCS pipeline. The corresponding AwA2 figures need no such note, at 0.8699 against 0.869.

5.4. What Carries the Method

The configurations in Table 5.1 isolate the contribution of each component. Removing the pairwise term is by far the most damaging single change, which locates the method's performance in its self-generated constraints rather than in its clustering objective or its descriptions. That finding then raises a question about what those constraints contain, and the answer differs sharply between the two datasets.

5.4.1 The Pairwise Objective

Setting $\lambda = 0$ removes the self-generated pairwise loss and leaves the mutual information objective and the ILP in place. On AwA2 this costs 0.18 NMI, from 0.9229 to 0.7425. On aPY-15 it costs 0.148, from 0.6798 to 0.5318. In both cases the result falls below the matched k-means baseline, so without its pairwise term the method does not merely weaken but ceases to be competitive with clustering the frozen features directly.

This is the clearest positive finding of the reproduction, and it is worth stating in the paper's favour: the self-generated constraint mechanism is doing real work, and it is doing nearly all of the work. The mutual information objective on its own, even with the description module attached, is not what produces the published performance.

5.4.2 On AwA2 the Constraints Are Effectively Class Labels

If the pairwise constraints carry the method, what they encode determines what the reported numbers mean. On AwA2 they encode the class labels almost exactly. The fraction of generated pairs joining two instances of the same true class is 0.9215 ± 0.0021, and that value is identical to four decimal places across every configuration in Table 5.1 and every seed. It is already at that level at the first pretraining epoch, before the network has learned anything, and it does not move as training proceeds.

The mechanism is a property of the dataset rather than of the method. AwA2 attributes are defined per class, so two instances of the same class have a tag distance of exactly zero, and the weight $\gamma = 100$ makes that term dominate the pair-selection score of Equation (5). Any same-class instance present among the annotated candidates is therefore selected in preference to any other, and the constraint set becomes a noisy copy of the class partition.

The measured value is what a simple counting argument predicts. With batch size 256 and $r = 0.5$, roughly 128 annotated instances per batch are spread across 50 classes, giving about 2.6 same-class candidates per anchor. The probability that an anchor finds none is approximately $e^{-2.6} \approx 0.074$, predicting a same-class fraction near 92.6% against the 92.15% measured. The residual is accounted for by batch-sampling misses rather than by any property of the learned representation, which confirms that the fraction is fixed by the mask and the tag space and not by training.

The tags-only control corroborates this independently. Clustering the masked tag matrix alone, with no image features at all, gives ACC = 0.4976, matching the annotated fraction 14,855/29,857 = 0.4975 to within 10^{-4}. At $r = 0.5$ the AwA2 tag matrix contains only 51 distinct rows, one per class plus the shared imputed row for unannotated instances, so the tag block recovers class identity perfectly for every annotated instance and carries no information about the rest. Its zero variance across seeds is structural for the same reason.

5.4.3 The Finding Is Dataset-Specific

The preceding result must be scoped carefully: it is a property of per-class-attribute datasets, not of the method in general. Table 5.2 contrasts AwA2 with aPY-15 evaluated on genuine per-instance annotations, recovered from the original files as described in Appendix C.4. Chapter 4 notes that the shipped preprocessing averages these into per-class means; for this chapter they were reconstructed at instance level, so that the constraint mechanism could be tested where tag distances are not degenerate.

Table 5.2: Dataset properties governing the self-generated constraint mechanism. AwA2 attributes are defined per class; the aPY-15 figures use per-instance binary annotations recovered from the original files (Appendix C.4).

Quantity	**AwA2**	**aPY-15 (per-instance)**
Tag density	0.365	0.0869
Distinct tag rows at $r = 0.5$	51	1,080–1,107 (1,703 raw)
Within-class tag distance	1.0 exactly zero	0.0332 raw; 0.0308–0.0372 annotated
Same-class pair fraction	0.9215 ± 0.0021	0.5202–0.6264
Tags-only k-means ACC	0.4976 (= 14,855/29,857)	0.3239 (annotated fraction 0.5093)
Max reachable coverage, Eq. 3.2	≈ 31	5.56 mean; 4.18–4.41 worst class

Every quantity that produces the AwA2 result is absent on aPY-15. There are 1,703 distinct tag rows rather than 51, and within-class tag distance is zero for only about 3% of same-class pairs rather than all of them. The same-class fraction accordingly sits between 0.52 and 0.63 depending on configuration, close to what an uninformed selection over 15 classes would produce, and it moves with the configuration rather than remaining pinned. The tags-only control does not leak: at ACC = 0.3239 against an annotated fraction of 0.5093, the tag block does not recover class identity even for the instances that have annotations.

The claim of Section 5.4.2 is therefore that on datasets whose attributes are defined per class, the original DDC's self-generated constraints reduce to class supervision. It is not a claim about the method on data with genuine per-instance annotations, where the constraints are informative but imperfect, as the $\lambda = 0$ result of Section 5.4.1 confirms.

5.5. The Description Channel

The pairwise term carries the method; the question that remains is whether the description module contributes to the clustering through it. This is the channel DDECCS deliberately omits, so it is the measurement most directly relevant to this thesis. The ILP influences clustering only through the mask g, which selects the tags used in constraint generation, and disabling the ILP while holding everything else fixed therefore isolates the channel exactly. Table 5.3 reports the paired per-seed differences on both datasets.

Table 5.3: Paired differences in NMI between the ILP-on and ILP-off configurations at $\mu = 1.5$, $\lambda = 1$, with all other settings and the seed held fixed. A negative value means the description channel degrades clustering.

Seed	**AwA2** ($\alpha = 3$)	**aPY-15** ($\alpha = 1$)
42	-0.0059	-0.0494
43	-0.0067	-0.0418
44	-0.0022	-0.0423
45	$+0.0042$	-0.0391
46	-0.0025	-0.0425
Mean	-0.0026	$\mathbf{-0.0430 \pm 0.0034}$
Negative	4 of 5	5 of 5

5.5.1 Inert on AwA2, and Structurally So

On AwA2 the channel does nothing. The mean paired difference is -0.0026, negative on four of five seeds, and smaller than the seed-to-seed standard deviation of NMI itself, which is about 0.006. Enabling or disabling the description module leaves the clustering statistically unchanged.

The reason is structural rather than empirical, and it follows from Section 5.4.2. The mask g can only remove tags from the distance computation. When two same-class instances have a tag distance of exactly zero, that distance remains exactly zero under every possible mask, because removing coordinates from a vector of zero differences cannot make it non-zero. The pair-selection score is therefore invariant to g for precisely the pairs that dominate the selection. This is confirmed directly: across the five seeds the ILP mask varied between 47 and 73 selected tags, while the same-class pair fraction remained 0.9215 ± 0.0021 throughout, with a correlation between the two of -0.07. The description module changes what the clusters are described by; it cannot change which pairs are generated.

5.5.2 Live but Harmful on aPY-15

On aPY-15, where tag distances are not degenerate, the channel is live and its sign is negative. The mean paired difference is -0.0430 ± 0.0034, negative on all five seeds, and roughly twelve times its own standard deviation. This is not noise, and it is an order of magnitude larger than the AwA2 effect.

The mechanism is visible in the constraint statistics. Masking removes tags from the distance computation, and on per-instance annotations those tags are discriminative, so pairs selected in the masked space are less likely to join same-class instances than pairs selected in the full space: the same-class fraction drops from 0.6233 with the ILP disabled to 0.5554 at $\alpha = 1$. The description module degrades the supervision it is meant to refine. The two mechanisms of the method thus pull in opposite directions on this dataset: the $\lambda = 0$ comparison shows the pairwise term is worth 0.148 NMI, while the mask costs 0.043 of that back.

5.5.3 Descriptions Are Best Where Clustering Is Worst

Because the damage is caused by masking, its magnitude should scale with how aggressively the ILP masks, and it does. Table 5.4 varies the coverage parameter on aPY-15 and reports both the clustering cost and the description quality at each setting.

Table 5.4: Effect of coverage parameter α on aPY-15, five seeds. ΔNMI is the paired difference against the ILP-off configuration. At $\alpha = 4$ the ILP is infeasible in every round, so no mask is applied and no descriptions are produced. At $\alpha = 3$ the TC, ITF and tags-per-cluster figures are over the two seeds for which the ILP was feasible on the final assignment. ITF values are in nats against a ceiling of $\ln 15 \approx 2.708$.

α	Tags kept	ΔNMI	Neg.	TC	ITF	Tags/cluster
1	26.4/64 (41%)	$\mathbf{-0.0430 \pm 0.0034}$	5/5	**0.3798**	**2.0853**	3.09 ± 0.14
2	47.8/64 (75%)	-0.0063 ± 0.0056	4/5	0.3056	1.7143	7.77 ± 0.45
3	50.5/64 (79%)	-0.0089 ± 0.0055	5/5	~ 0.311	~ 1.157	12.13 ± 0.60
4	none (infeasible)	-0.0031 ± 0.0061	1/5	—	—	—

The dose-response is clean and it is inverted with respect to description quality. At $\alpha = 1$ the mask is most aggressive, keeping 41% of tags, and produces both the largest clustering penalty and by some margin the best descriptions: 3.09 tags per cluster, TC = 0.3798, and ITF = 2.0853 against a ceiling of $\ln 15 \approx 2.708$. At $\alpha = 2$ the penalty all but disappears, and the descriptions become markedly worse on both metrics while more than doubling in length. At $\alpha = 4$ the ILP is infeasible in every round, no mask is applied, and the runs are effectively identical to disabling the description module, which serves as an internal control confirming that the effect is caused by masking and nothing else.

The paper's own ablation asserts that the description module consistently improves clustering. That does not replicate here. On AwA2 the channel is inert for a reason that follows from the dataset's attribute structure; on aPY-15 it is active and harmful at every feasible operating point; and the settings that produce the most concise and distinctive descriptions are the settings that damage the clustering most. We report this as a failure to replicate under the conditions of Section 5.2.2, not as a claim about the authors' own runs, which we cannot inspect.

5.6. Feasibility of the Published Configuration

Two aspects of the published setup could not be reproduced as described. The first concerns the coverage parameter on aPY and is arithmetic rather than empirical. The second and third are smaller discrepancies in reported cost and in the behaviour of one of the description metrics.

5.6.1 The Published Coverage Parameter on aPY

The paper states a single global coverage value, $\alpha = 8$, for both datasets, and defines Q_{ij} as the fraction of cluster i carrying tag j under mean imputation, which is the definition implemented in Section 3.5. Under those two statements together, the coverage constraint of Equation 3.2 cannot be satisfied on aPY at any β. The left-hand side is bounded above by the row sum of Q, which is the mean number of active attributes per cluster member. On the true aPY-15 partition that quantity is 5.56 on average and between 4.18 and 4.41 for the worst class, so selecting every one of the 64 attributes still leaves the constraint unmet. Learned partitions, being less pure, reach less. In practice $\alpha = 4$ was infeasible in every round we ran, $\alpha = 3$ was feasible in about 37% of rounds, and only $\alpha = 1$ and $\alpha = 2$ solved reliably.

This is a discrepancy that we cannot resolve. Something in the paper's aPY setup must differ from what is described — a different normalization of Q, a different binarization threshold, or a denser subset of the data — but with no reference implementation available there is no way to determine which, and we report it as an unresolved difference rather than as an error. It is worth noting that the paper's own analysis of α observes independently that smaller values yield more concise and more orthogonal explanations, which is consistent with what Table 5.4 shows.

The consequence for this thesis is direct. The adaptive rule of Section 3.5.2 yields $\alpha = 2$ on aPY. That value is feasible, solves to proven optimality at $\beta = 1$, and, from the dose-response, sits almost exactly at the point where the mask stops harming the clustering while descriptions remain usable. What Chapter 3 introduced as a fix for an infeasible ILP turns out to be what makes the original method executable on this dataset at all, and it lands close to the best available operating point rather than merely a feasible one.

5.6.2 Two Further Discrepancies

The paper reports that solving the ILP accounts for roughly 1% of training time. In the reproduction it dominated. On AwA2 at $\mu = 1.5$, a complete run took 529 seconds with the ILP enabled and 58 seconds with it disabled and everything else unchanged, so the description module accounted for about 89% of wall-clock time. The gap is large enough that it is unlikely to be explained by hardware or solver version alone, though we cannot rule those out; the per-configuration runtimes are in Appendix C.7. The practical consequence is that the description module is the expensive component of the method, which is relevant to DDECCS given that it solves the same ILP once rather than once per round.

The second concerns Tag Coverage as a measure. On AwA2 the $\mu = 1.5$ configuration reaches TC = 0.7538 at NMI = 0.9229, while removing the pairwise term drops NMI to 0.7425 but leaves TC at 0.7503 — a difference of 0.0035 for a difference of 0.18 in clustering quality. Both figures come from the same solver budget and are therefore directly comparable. A description-quality metric that cannot distinguish a 0.92 clustering from a 0.74 one is measuring something close to the marginal attribute statistics of the dataset rather than the fit between a description and its cluster, and TC values should be read with that limitation in mind, in this chapter and in Chapter 4 alike. ITF separates the two configurations only slightly better, at 2.4563 against 2.4087. Neither metric is comparable with Chapter 4 in any case, for the reasons given in Section 5.2.2.

5.7. Implications for DDECCS

The purpose of this chapter was to supply the controlled comparison Chapter 4 could not. Three consequences follow for the method this thesis proposes, and each concerns a design choice DDECCS had already made for other reasons.

The first concerns what the clustering numbers on either side actually measure. DDECCS applies the ILP once, after clustering, so the semantic attributes never enter the partition; its NMI, ACC and ARI are produced by frozen features and unsupervised algorithms alone. On AwA2 the original DDC's are not, because its self-generated constraints reproduce the class partition at 92.15% before training begins. The two

sets of figures are therefore not measuring the same thing on per-class-attribute data, and the comparison in Table 4.6 should be read in that light. Chapter 4 presents the post-hoc design as a limitation, in that the ILP cannot influence cluster assignments. On this evidence the same property is better described as a correctness guarantee: it is what makes the reported clustering quality attributable to the clustering.

The second concerns the explanation for the remaining gap. The matched comparison is the first evidence in this thesis that bears on it directly, and on both datasets it points away from feature refinement. On AwA2 a substantial part of the margin comes from class information entering through the pairwise term rather than from better features, and the paper-faithful configuration in fact loses to a matched k-means baseline. On aPY the faithful method does not reach the published figure at any feasible coverage value, and the reproduction sits below the frozen-feature baseline throughout. Feature adaptation may still contribute, but it is not the whole of the difference, and on aPY it is not demonstrated at all under matched conditions.

The third concerns the integration the title of this thesis proposes. DDECCS pairs the two methods post-hoc; a fully trained integration would run the description module's output back into the clustering, through exactly the mask g measured in Section 5.5. That channel was built and measured on both datasets. It is inert on one, for reasons that follow from the attribute structure rather than from any implementation choice, and it is reliably harmful on the other, with the settings producing the best descriptions being the ones that damage the clustering most. Building the trained integration on that channel was therefore not pursued, and the reason is a measurement rather than a limitation of effort or scope. This is what the reproduction contributes to the thesis's central question: the description-to-clustering feedback loop, on these two datasets, does not carry value, and the post-hoc design forgoes nothing that was demonstrably there to gain. Chapter 6 revisits the research questions in light of this.

6. Discussion

This chapter interprets the experimental results in light of the research questions. We first explain when and why consensus clustering helps, assess the quality of the generated descriptions, and account for the gap to DDC (Section 6.1). We then analyse why the trained alternatives we attempted failed (Section 6.2) and what trade-offs the final design entails (Section 6.3), and revisit the three research questions (Section 6.4).

6.1. Interpretation of Results

The experiments support two headline findings. First, DECCS consensus delivers its largest gains precisely where individual algorithms struggle, while adding little where the features already separate the classes. Second, the post-hoc ILP produces concise, distinctive descriptions across both datasets without affecting clustering quality. We examine each finding in turn, then account for the gap to the original DDC.

6.1.1 When Does Consensus Clustering Help?

Our results reveal a clear pattern: DECCS consensus provides significant improvements when the underlying features do not perfectly separate classes, but offers diminishing returns when features are already highly discriminative.

On AwA2, ResNet-101 features separate the 50 animal classes so effectively that K-means alone achieves NMI = 0.869. DECCS consensus matches this performance (NMI = 0.871) but does not substantially improve it. The four base clusterers (K-means, Spectral, GMM, Agglomerative) largely agree because the feature space has clear class structure, so ensembling their decisions adds little information. However, DECCS does provide measurably higher ACC (0.807 vs. 0.785) and roughly three- to sevenfold lower variance across seeds, indicating more consistent and accurately aligned cluster assignments.

On aPY, the picture changes dramatically. The 32-class aPY dataset contains diverse object categories—animals, vehicles, furniture, containers—that are less cleanly separated in ResNet feature space. Here, K-means achieves only NMI = 0.534 with substantial variance ($\pm$0.014), while DECCS consensus reaches NMI = 0.618 ($\pm$0.006), an improvement of 15.6%. This demonstrates that consensus clustering serves as a regularizer: when individual algorithms make different errors (K-means is sensitive to initialization, Spectral clustering captures different structure than GMM), their agreement signal identifies robust cluster boundaries. The reduced NMI variance is valuable in practice: the amount of class information recovered is largely independent of random initialization. The partition itself is not correspondingly stabilized on this dataset, as the higher ACC and ARI variance shows: NMI is invariant to label permutation and tolerant of cluster-size imbalance, whereas ARI counts agreeing pairs

and ACC depends on an optimal-assignment matching that can switch between near-equivalent alternatives. Section 4.4.4 rules out preprocessing as the source of the NMI gain.

One metric runs counter to this improvement, and it is worth addressing directly. The Silhouette score—our only label-free measure—falls under consensus on AwA2 (0.189 to 0.160) and turns slightly negative on aPY-32 (0.013 to −0.018), even though NMI, ACC, and ARI improve or hold. This is expected rather than contradictory. Silhouette rewards clusters that are compact and well-separated in Euclidean feature space, whereas the final DECCS step applies spectral clustering to the co-association graph, which optimizes a normalized graph cut rather than Euclidean compactness. A consensus partition can therefore agree more closely with the true classes (higher NMI/ACC) while being less Euclidean-compact (lower Silhouette). Because our objective is agreement with the semantic classes, we treat the label-based metrics as primary and report Silhouette only as a label-free diagnostic rather than as the arbiter of cluster quality.

6.1.2 Quality of ILP-Generated Descriptions

The ILP produces descriptions that are both concise and semantically meaningful. On AwA2, the tiger cluster (in `ddc` mode) is described by "orange, stripes, muscle, fierce" (TC = 0.82, ITF = 3.12), meaning these attributes have a mean value of 0.82 across the cluster's members and the attributes are highly distinctive to this cluster. On aPY, the orthogonality constraint keeps descriptions distinctive: the car cluster ("Wheel, Door, Taillight, Side mirror, Exhaust") and the bicycle cluster ("Pedal, Handlebars, Text, Plastic") are described by disjoint attribute sets, even though both are vehicles.

Across all datasets and clusters, Tag Coverage ranges from 0.18 to 0.87 and Inverse Tag Frequency reaches up to 3.47, confirming that the ILP generates descriptions that are representative of their clusters and distinctive across them. The lower TC values occur on aPY, where sparse class-level attributes and visually mixed clusters dilute coverage; the higher values occur on AwA2 and on the cleaner aPY clusters.

The adaptive α parameter (Section 3.5.2) was essential for obtaining ILP solutions on aPY at all: DDC's default $\alpha = 8$ is infeasible on aPY's sparse attributes, and our density-proportional scaling restores feasibility while keeping descriptions concise. Without it the description stage does not merely degrade but fails outright on aPY, which is what makes DDC's ILP usable beyond the dense-attribute setting it was designed for.

The comparison to the original DDC's reported description metrics (Table 4.8) must be read with care. Three differences make a direct comparison unsafe. First, scale: DDC reports description metrics only on a down-sampled setting of 10 AwA classes merged into 5 clusters, each a clean pair of classes, whereas we describe 50 clusters of the full AwA2; coverage and distinctiveness are both far easier to maximise over 5 well-separated clusters than over 50. Second, the coverage parameter differs ($\alpha = 8$ versus our adaptive $\alpha = 3$): more tags per cluster raises TC and, through overlap, lowers ITF, so the two methods occupy different points on the coverage–distinctiveness trade-off by construction. Third, the ITF logarithm base differs ($\log_2$ versus natural log). Fourth, the attribute representation differs: DDC's tags are binary, so its Q is a genuine within-cluster fraction, whereas ours is a mean over continuous per-class values. The decisive point, though, is structural. Each of DDC's five clusters is a clean pair of AwA classes, and AwA attributes are shared by every instance of a class, so

any tag true of both constituent classes is true of 100% of the cluster by construction. TC = 1.00 there measures the evaluation design, not the description module. DDC's published figures accordingly cannot settle whether iterative ILP-in-training produces better descriptions than our post-hoc ILP, in either direction. A controlled comparison would require running both methods on the same clustering at the same α and over the same attribute representation. Chapter 5 supplies the first two conditions but not the third, since the reproduction operates on binary attributes rather than the continuous ones used here; a description-quality comparison holding all three fixed remains for future work.

The most distinctive cluster is invariant to the clustering method: the potted-plant cluster attains the maximum possible ITF ($\ln 32 \approx 3.47$) in both `ddc` and `ddeccs` modes, indicating that the consensus step preserves the descriptions' distinctiveness rather than eroding it.

6.1.3 Why Our Approach Trails DDC

The original DDC outperforms our pipeline, and the reason is consistent across both datasets: DDC refines its features for the clustering task, whereas we do not. Our K-means baseline exceeds DDC's reported K-means baseline on both datasets (0.869 vs. 0.742 on AwA, 0.666 vs. 0.644 on aPY-15), but this comparison is not direct, and not only because DDC uses $K = 40$ on the original AwA (19,832 instances) while we use $K = 50$ on AwA2 (29,857 instances). The paper describes its baseline as K-means on the image features naively concatenated with the semantic attributes, computed on the same pre-trained ResNet-101 features it uses for its other non-deep baselines [Zhang and Davidson, 2021, Sec. 4.4], whereas ours clusters the ResNet features alone. That difference does not explain the margin: Chapter 5 runs the same construction on our data and finds that concatenating the attributes *raises* K-means rather than lowering it. Their baseline stands at 0.742 where the same construction here reaches 0.880, on comparable but not identical data, and with no reference implementation available the source of that difference cannot be determined. The more meaningful comparison is between the full methods. DDC's trained MLP (NMI = 0.896) outperforms our best frozen-features result (DDECCS, NMI = 0.871) by about 2.5 points, though on different data (AwA1 versus AwA2).

On aPY the gap is larger: DDC achieves NMI = 0.863 while our best result (DDECCS) is 0.684, a gap of 17.9 points. Feature refinement is a plausible part of the explanation—because aPY images are bounding-box object crops rather than full scenes, ImageNet-pretrained ResNet features are less discriminative on aPY than on AwA2, so task-specific refinement would add more value there—but it is not the whole of it. Under the matched conditions of Chapter 5, the reproduced method does not reach the published aPY figure at any feasible coverage parameter, and on AwA2 a substantial part of its margin comes from class information entering through its pairwise term rather than from better features. Section 5.7 sets out what the controlled comparison does and does not attribute to feature training.

Two caveats temper this comparison. First, DDC reports results on the original AwA dataset, which has been withdrawn for copyright reasons; we use the replacement AwA2, with different images, so the AwA comparison is approximate. Second, DDC trains with a tag annotation ratio $r = 0.5$ (half of each instance's attributes randomly masked). Our own MLP experiments (Section A.2.2) found this per-instance masking

critical for making pairwise constraints do anything at all, though our masking deviated from DDC's (Appendix A.2), so this is consistent with rather than confirmation of its role in DDC's framework.

The experimental setup matters as much as the algorithm here: filtering to DDC's actual 15 classes, rather than forcing all 32 into 15 clusters as in our initial experiments, raised K-means NMI to 0.666 on its own.

6.2. Why the Trained Approaches Failed

A central empirical finding of this thesis is negative: every attempt to train a feature representation underperformed simple K-means on frozen features. Understanding why this happened is what justified the final design.

We first tried a convolutional autoencoder with a tag-prediction branch, then DDC's own objective (mutual information maximization with self-generated pairwise constraints) using a 3-layer MLP on frozen features. The MLP approach failed in a specific, diagnosable way: on the full dataset the MI loss dominated from the very first epoch, reaching $L_{MI} = -0.92$ after a single epoch of 116 mini-batches on 29857 samples and committing the model to roughly 24 coarse clusters before the pairwise constraints ($L_P \approx 0.5$) could exert any influence. On a small 1,600-instance subset (6 mini-batches per epoch), both losses started near zero and grew together, allowing the pairwise term to shape cluster formation—but this behaviour did not survive at scale.

The decisive observation was that K-means on raw 2048-dimensional ResNet features achieves NMI = 0.869 on AwA2, whereas every trained variant collapsed at full scale (NMI $\leq$ 0.012). The pretrained backbone already encodes the class-discriminative structure these datasets require, and our training runs degraded that structure rather than refining it. A subsequent code review (Appendix A.2) found that the trained variants also contained implementation defects—most notably a consensus term that carried no gradient and an ILP constraint-selection step that was never invoked during training—so the negative result is properly scoped: our trained variants could not improve on the frozen features, while the question of whether a fully faithful implementation could remains open. What the result does establish, together with the standardization ablation (Section 4.4.4), is that the frozen features are a strong and easily degraded starting point, which is what redirected the work toward frozen features plus consensus and post-hoc descriptions.

6.3. Trade-offs of the Frozen-Features Design

The frozen-features design buys reliability at the cost of adaptivity, and this trade-off explains the pattern of results.

On the positive side, the pipeline is reproducible. DECCS consensus reduces run-to-run NMI variance several-fold relative to K-means, and because no training is involved there are no loss-balancing instabilities, no learning-rate sensitivity, and no risk of the optimization collapsing into a degenerate solution—all problems we encountered in the trained variants. Feature extraction is also a one-time cost, cached to disk, so the full sweep of experiments is cheap to repeat.

On the negative side, the representation cannot adapt to the task. Where the pretrained features do not naturally separate the target classes—most clearly on aPY's cropped object categories—there is no mechanism to improve them, which is the

proximate cause of the gap to DDC. A second, structural trade-off concerns the descriptions: because the ILP runs post-hoc on a fixed clustering, the descriptions explain the clusters but cannot feed back to improve them. In DDC's iterative loop, better descriptions yield better-filtered pairwise constraints, which yield better clusters; our design forgoes that virtuous cycle in exchange for the ability to evaluate clustering and description quality independently. The trade-off is therefore favorable on well-separated data (AwA2), where adaptivity buys little, and unfavorable on harder data (aPY), where it would buy more.

6.4. Research Questions Revisited

We now return to the three research questions posed in the introduction and summarize how the thesis answers each.

6.4.1 RQ1: How can DDC's interpretability be integrated into DECCS?

We integrate DDC's interpretability through the ILP-based description module, which generates concise, orthogonal cluster descriptions from semantic attributes. The integration is post-hoc: the ILP operates on the cluster assignments produced by DECCS consensus rather than being embedded in a training loop. The resulting descriptions (in DDECCS mode) are meaningful—average TC = 0.70 on AwA2 and 0.57 on the 32-class aPY—and distinctive—average ITF = 2.66 and 2.44 respectively—while the ILP itself solves in seconds to about a minute.

6.4.2 RQ2: What impact does the integration have on clustering performance?

The ILP description module has zero impact on clustering performance: it operates post-hoc and does not modify cluster assignments. The DECCS consensus module, by contrast, improves clustering substantially on aPY (+15.6% NMI over K-means) and matches K-means on AwA2 with improved accuracy and stability. Our pipeline achieves NMI = 0.871 on AwA2 (vs. the original DDC's 0.896) and 0.684 on aPY-15 (vs. DDC's 0.863). That remaining gap is not primarily a matter of feature refinement: under the matched conditions of Chapter 5, much of the original DDC's AwA2 margin comes from class information entering through its self-generated pairwise constraints, and its paper-faithful configuration loses to a matched K-means baseline on both datasets (Sections 6.1.3 and 5.7).

6.4.3 RQ3: How does the approach perform on AwA2 and aPY?

On AwA2, the approach achieves strong clustering (NMI = 0.871, within 2.5 points of DDC's reported 0.896) with interpretable descriptions, reflecting the quality of pretrained ResNet-101 features for animal classification. On aPY it is less competitive with DDC (NMI = 0.684 vs. 0.863) but demonstrates that DECCS consensus provides substantial value over K-means (+15.6% on the full 32-class dataset). The diverse,

cropped object categories make aPY inherently harder for a frozen-features approach. Whether task-specific training would close that gap is not established: under the matched conditions of Chapter 5 the reproduced method does not reach the published aPY figure at any feasible coverage parameter, and with no reference implementation available the source of that discrepancy cannot be determined (Section 5.7).

7. Conclusion

This chapter concludes the thesis. It summarizes the contributions and main results, distils the notable insights gained during the work, states the limitations and the scope of what was done, and outlines directions for future research.

7.1. Summary of Contributions

This thesis investigated the integration of Deep Descriptive Clustering (DDC) interpretability mechanisms into the DECCS consensus clustering framework, with evaluation on the AwA2 and aPY attribute-annotated image datasets. The resulting framework, DDECCS, combines pretrained ResNet-101 feature extraction, ensemble-based consensus clustering, and Integer Linear Programming for generating interpretable cluster descriptions.

The work makes five contributions. First, the DDECCS framework itself—the first pipeline to pair DECCS-style consensus clustering with DDC-style ILP descriptions on frozen pretrained features—which provides robust, reproducible clustering together with human-readable cluster descriptions, and shows that consensus clustering improves substantially over K-means on challenging multi-category data (+15.6% NMI on aPY) while matching it on well-separated data (AwA2) with better stability and accuracy. Second, the adaptive coverage parameter α, a density-proportional rule that restores ILP feasibility on sparse binary attributes where DDC's fixed $\alpha = 8$ fails. Third, the aPY preprocessing pipeline and 15-class subset, which reconstructs DDC's class selection from the raw annotations to make the comparison as close as the published information allows—and itself raised K-means NMI to 0.666 over forcing all 32 classes into 15 clusters. Fourth, a systematic empirical study across two datasets, four clustering modes, and ten random seeds that establishes reproducible baselines and documents in detail the trained approaches that failed and why. Fifth, a controlled reproduction of the original DDC, built from its published specification because no public implementation exists, and evaluated against matched baselines on both datasets—establishing that its self-generated constraints reduce to class supervision on per-class-attribute data, that its description-to-clustering channel does not improve clustering on either dataset, and that its published coverage parameter is unreachable on aPY.

7.2. Notable Insights

We distil three transferable insights from the work, beyond the specific contributions above.

7.2.1 Pretrained Features Can Be Sufficient

One of the most relevant findings was that K-means on raw ResNet-101 features (NMI = 0.869 on AwA2) outperformed every trained model we built. Modern pretrained backbones encode enough class-discriminative structure that a simple baseline on frozen features is hard to beat, and easy to damage: our trained variants degraded it, though a code review found implementation defects in them (Appendix A.2), so this bounds our implementations rather than trained approaches in general. DDC's published results show that a correctly trained representation does improve on frozen features. The transferable lesson is narrower: frozen-feature K-means deserves to be reported as a baseline before a trained representation is claimed to be necessary.

7.2.2 Loss Balancing in Deep Clustering Is Fragile

Our extensive experiments with DDC-style MLP training revealed that mutual information maximization dominates pairwise constraint losses on large datasets. A fixed loss weight λ that at least allows the pairwise term to act at one scale (1,600 samples) fails at another (29857 samples), because the relative magnitudes of the loss terms change with dataset size and the number of mini-batches per epoch. This fragility suggests that adaptive loss balancing techniques [Crawshaw, 2020] may be necessary for scaling deep descriptive clustering.

7.2.3 Implementation Details Matter

The discovery that DDC's per-instance tag masking ($r = 0.5$) was essential for pairwise constraints (without it, same-class instances had identical tags, making pairwise selection ignore tag information) highlights how crucial implementation details are for reproducing published results—as does the fact that our own masking, re-drawn per batch rather than once, deviated from DDC's and undermined the very mechanism it was meant to enable (Appendix A.2).

7.3. Limitations

The most significant limitation is the absence of task-specific feature learning. DDC's MLP training with mutual information maximization and pairwise constraints refines features for the clustering task, whereas our frozen-features approach relies entirely on the pretrained backbone's general-purpose representations, and on aPY in particular those features do not naturally separate the cropped object categories. The controlled comparison of Chapter 5 bounds how much of the gap to DDC this explains. On AwA2, a substantial part of DDC's margin comes from class information entering through its self-generated pairwise constraints, which reproduce the class partition at 92% before training begins; the configuration faithful to the published objective in fact loses to a matched K-means baseline. On aPY, the reproduced method does not reach the published figure at any feasible coverage parameter. Absent feature adaptation therefore remains a real limitation of the design, but it is not by itself the explanation for the published gap.

A second, structural limitation is that the post-hoc ILP cannot improve clustering: it generates the best description for a fixed partition but has no path back to refining it. Section 6.3 examines the trade-off this buys.

Third, the approach depends on the quality of the pretrained features. Our results are bounded by what ResNet-101 captures; for domains where ImageNet pretraining is less relevant—such as medical or satellite imagery—the pipeline would require domain-specific pretrained backbones. In addition, the ILP requires per-class or per-instance semantic attribute annotations, which are not available for most datasets, and on both datasets the ILP sees one attribute vector per class—inherently on AwA2, and by our own averaging of aPY's per-instance annotations (Section 4.2.2)—which limits its ability to describe within-class variation. On aPY this is a choice we made for uniformity rather than a property of the data, and re-running the ILP on the raw per-instance vectors is a straightforward extension.

Finally, the adaptive α is validated at only a few operating points. On the three datasets evaluated it produces just two distinct values—$\alpha = 3$ for AwA2 and $\alpha = 2$ for both aPY variants—and on the sparsest (aPY-15) the lower clip bound rather than the density term determines the value ($16 \times 0.087 \approx 1.4$, clipped to 2). The continuous density-proportional scaling is therefore demonstrated to restore feasibility but not validated across a range of densities; a systematic sweep to confirm the $16\bar{d}$ relationship is left to future work.

7.4. Scope and Delimitations

We state explicitly what this thesis did and did not address, to delimit the claims made above.

7.4.1 In Scope

This thesis covered the integration of DDC's ILP-based descriptions with DECCS consensus clustering; evaluation on AwA2 (50 classes, 85 attributes) and aPY (32 classes and the 15-class DDC subset, 64 attributes); comparison with DDC's published results; and documentation of the alternative approaches attempted.

7.4.2 Out of Scope

An early attempt at DDC's trained-encoder approach—an MLP on frozen ResNet features with mutual-information and pairwise losses—did not scale to the full datasets and is documented as part of the early experiments (Section 4.7). A subsequent paper-faithful reproduction of the original method, evaluated against matched baselines on both datasets, is reported in Chapter 5; what remained out of scope was integrating it into a jointly trained pipeline, which Section 5.7 explains was not pursued because the channel such an integration would run through was measured and found not to carry value. DDC evaluates at tag annotation ratios $r\% \in \{10, 30, 50\}$, where each instance has a fraction of its attributes randomly masked to simulate incomplete annotation; our pipeline uses full, unmasked attributes, since we do not train with the pairwise constraints that require masking. Natural language explanation generation beyond attribute lists, and evaluation on datasets other than AwA2 and aPY, were also left outside the scope of this work.

7.5. Future Work

Several directions extend naturally from the framework and the open problems it surfaced.

7.5.1 Resolving the MLP Training Challenge

The core technical challenge—making DDC's MLP training work at scale—remains open. Potential directions include adaptive loss balancing such as GradNorm [Chen et al., 2018], learning-rate warmup schedules that delay MI convergence, and curriculum-based training that gradually increases dataset size. Surveys of multi-task optimization [Crawshaw, 2020] catalogue several such balancing strategies.

7.5.2 End-to-End Consensus with Descriptions

Integrating the ILP into a training loop where descriptions inform consensus construction could create the iterative refinement cycle that our post-hoc approach lacks. This would require making the discrete ILP solvable within a gradient-based training loop, which remains an open research problem.

7.5.3 Extension to Other Domains

The DDECCS framework is applicable to any domain with pretrained feature extractors and semantic attribute annotations. Natural extensions include medical image clustering with clinical attributes, scientific image analysis with domain-specific descriptors, and text clustering with topic-based attributes.

7.6. Final Remarks

This thesis demonstrates that interpretable clustering does not require sacrificing performance. By combining the strengths of consensus clustering (robustness, stability) with ILP-based descriptions (conciseness, orthogonality), the DDECCS framework provides both accurate clusters and human-understandable explanations. The journey from initial autoencoder experiments through MLP training challenges to the final frozen-features pipeline illustrates that in deep learning research, the path to a working solution is not linear, and documenting what did not work is as valuable as reporting what did.

Bibliography

D. Bertsimas, A. Orfanoudaki, and H. Wiberg. Interpretable clustering: An optimization approach. *Machine Learning*, 110(1):89–138, 2021.

M. Caron, P. Bojanowski, A. Joulin, and M. Douze. Deep clustering for unsupervised learning of visual features. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 132–149, 2018.

J. Chang, L. Wang, G. Meng, S. Xiang, and C. Pan. Deep adaptive image clustering. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 5879–5887, 2017.

Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich. GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2018.

M. Crawshaw. Multi-task learning with deep neural networks: A survey. *arXiv preprint arXiv:2009.09796*, 2020.

Ian Davidson, Antoine Gourru, and S. S. Ravi. The cluster description problem - complexity results, formulations and approximations. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS'18, page 6193–6203, Red Hook, NY, USA, 2018. Curran Associates Inc.

J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.

A. Farhadi, I. Endres, D. Hoiem, and D. Forsyth. Describing objects by their attributes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1778–1785, 2009.

A. L. N. Fred and A. K. Jain. Combining multiple clusterings using evidence accumulation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(6): 835–850, 2005.

R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi. A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5): 1–42, 2018.

X. Guo, L. Gao, X. Liu, and J. Yin. Improved deep embedded clustering with local structure preservation. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1753–1759, 2017.

K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

L. Hubert and P. Arabie. Comparing partitions. *Journal of Classification*, 2(1):193–218, 1985.

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.

H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955.

C. H. Lampert, H. Nickisch, and S. Harmeling. Attribute-based classification for zero-shot visual object categorization. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 36(3):453–465, 2014. Original AwA dataset, now withdrawn due to copyright.

Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

S. P. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

D. Mautz, C. Plant, and C. Böhm. DeepECT: The deep embedded cluster tree. *Data Science and Engineering*, 5(4):419–432, 2020.

Lukas Miklautz, Martin Teuffenbach, Pascal Weber, Rona Perjuci, Walid Durani, Christian Böhm, and Claudia Plant. Deep clustering with consensus representations. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pages 1119–1124, 2022. doi: 10.1109/ICDM54844.2022.00141.

E. Min, X. Guo, Q. Liu, G. Zhang, J. Cui, and J. Long. A survey of clustering with deep learning: From the perspective of network architecture. *IEEE Access*, 6:39501–39514, 2018.

S. Mitchell, M. O'Sullivan, and I. Dunning. PuLP: A linear programming toolkit for Python. `https://github.com/coin-or/pulp`, 2011. Open-source LP/ILP solver interface for Python.

M. Moshkovitz, S. Dasgupta, C. Rashtchian, and N. Frost. Explainable k-means and k-medians clustering. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2020.

Y. Ren, J. Pu, Z. Yang, J. Xu, G. Li, X. Pu, P. S. Yu, and L. He. Deep clustering: A comprehensive survey. *IEEE Transactions on Neural Networks and Learning Systems*, 2024. Early Access.

M. T. Ribeiro, S. Singh, and C. Guestrin. Why should i trust you?: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.

P. J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987. Foundational metric paper.

P. Sambaturu, A. Gupta, I. Davidson, S. S. Ravi, A. Vullikanti, and A. Warren. Efficient algorithms for generating provably near-optimal cluster descriptors for explainability. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020. URL `https://ojs.aaai.org/index.php/AAAI/article/view/5462`.

A. Strehl and J. Ghosh. Cluster ensembles – a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research*, 3:583–617, 2002.

S. Vega-Pons and J. Ruiz-Shulcloper. A survey of clustering ensemble algorithms. *International Journal of Pattern Recognition and Artificial Intelligence*, 25(3):337–372, 2011.

Y. Xian, C. H. Lampert, B. Schiele, and Z. Akata. Zero-shot learning — a comprehensive evaluation of the good, the bad and the ugly. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(9):2251–2265, 2019. AwA2 dataset introduced as replacement for AwA.

J. Xie, R. Girshick, and A. Farhadi. Unsupervised deep embedding for clustering analysis. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 478–487, 2016.

J. Yang, D. Parikh, and D. Batra. Joint unsupervised learning of deep representations and image clusters. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5147–5156, 2016.

H. Zhang and I. Davidson. Deep descriptive clustering. In *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3342–3348, 2021. URL `https://arxiv.org/abs/2105.11549`.

S. Zhou, H. Xu, Z. Zheng, J. Chen, Z. Li, J. Bu, J. Wu, X. Wang, W. Zhu, and M. Ester. A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions. *ACM Computing Surveys*, 57(3):1–38, 2025. doi: 10.1145/3689036.

List of Figures

List of Tables

A. Complete Results, Early Experiments, and Solver Configuration

This appendix reports the individual per-seed results underlying the aggregated tables in Chapter 4, then documents the early experiments that motivated the final design and the ILP solver configuration. Recall that `ddc` reuses the `kmeans` clustering and `ddeccs` the `deccs` clustering, so each seed yields two distinct partitions; TC and ITF are reported for the two description modes.

A.1. Per-Seed Results

Tables A.1–A.3 give the individual results for all ten seeds on each dataset, with the mean and standard deviation that appear in the aggregated tables of Chapter 4.

Table A.1: Per-seed AwA2 results ($K = 50$, 29,857 samples). The TC/ITF columns belong to the `ddc` and `ddeccs` description modes respectively.

	kmeans / ddc					**deccs / ddeccs**				
Seed	NMI	ACC	ARI	TC	ITF	NMI	ACC	ARI	TC	ITF
42	0.8725	0.8038	0.7617	0.669	2.652	0.8712	0.8069	0.7362	0.698	2.656
43	0.8666	0.7748	0.7336	0.691	2.636	0.8710	0.8070	0.7365	0.695	2.656
44	0.8702	0.7826	0.7494	0.673	2.665	0.8721	0.8076	0.7404	0.698	2.671
45	0.8716	0.7895	0.7508	0.697	2.649	0.8714	0.8076	0.7393	0.699	2.643
46	0.8686	0.7816	0.7387	0.693	2.658	0.8708	0.8066	0.7370	0.699	2.661
47	0.8681	0.7743	0.7328	0.681	2.662	0.8723	0.8073	0.7385	0.699	2.658
48	0.8686	0.7831	0.7393	0.689	2.648	0.8715	0.8073	0.7376	0.694	2.653
49	0.8702	0.7865	0.7501	0.664	2.655	0.8715	0.8075	0.7376	0.698	2.647
50	0.8694	0.7871	0.7431	0.673	2.658	0.8719	0.8116	0.7385	0.695	2.654
51	0.8677	0.7892	0.7430	0.676	2.635	0.8706	0.8042	0.7372	0.702	2.656
Mean	0.869	0.785	0.744	0.681	2.652	0.871	0.807	0.738	0.698	2.656
Std	0.002	0.008	0.008	—	—	0.001	0.002	0.001	—	—

Table A.2: Per-seed aPY results ($K = 32$, 12,271 samples).

	kmeans / ddc					deccs / ddeccs				
Seed	NMI	ACC	ARI	TC	ITF	NMI	ACC	ARI	TC	ITF
42	0.5360	0.3964	0.1459	0.471	2.223	0.6102	0.5470	0.2528	0.564	2.430
43	0.5162	0.3687	0.1225	0.555	2.094	0.6194	0.6081	0.3436	0.571	2.461
44	0.5514	0.4080	0.1488	0.587	2.197	0.6239	0.6081	0.3362	0.593	2.459
45	0.5082	0.3600	0.1141	0.574	2.139	0.6138	0.5444	0.2509	0.578	2.418
46	0.5340	0.3877	0.1418	0.434	2.147	0.6257	0.6135	0.3441	0.566	2.449
47	0.5267	0.3754	0.1336	0.421	2.067	0.6128	0.5489	0.2551	0.565	2.400
48	0.5426	0.4027	0.1443	0.475	2.244	0.6128	0.5305	0.2354	0.584	2.475
49	0.5308	0.4014	0.1342	0.410	2.051	0.6120	0.5493	0.2621	0.578	2.444
50	0.5420	0.3939	0.1439	0.588	2.157	0.6250	0.6162	0.3479	0.577	2.411
51	0.5553	0.4081	0.1539	0.455	2.242	0.6190	0.6097	0.3428	0.560	2.450
Mean	0.534	0.390	0.138	0.497	2.156	0.618	0.578	0.297	0.574	2.440
Std	0.014	0.016	0.012	—	—	0.006	0.034	0.046	—	—

Table A.3: Per-seed aPY 15-class subset results ($K = 15$, 5,189 samples).

	kmeans / ddc					deccs / ddeccs				
Seed	NMI	ACC	ARI	TC	ITF	NMI	ACC	ARI	TC	ITF
42	0.6238	0.6057	0.4202	0.405	1.934	0.6769	0.6442	0.4588	0.492	2.245
43	0.6516	0.6633	0.4607	0.450	2.087	0.6870	0.6660	0.4852	0.494	2.220
44	0.6643	0.5741	0.4288	0.447	2.020	0.6800	0.6458	0.4670	0.492	2.235
45	0.6689	0.5620	0.4326	0.446	2.014	0.6867	0.6643	0.4813	0.493	2.228
46	0.6228	0.6007	0.4068	0.451	2.042	0.6879	0.6656	0.4857	0.482	2.219
47	0.7044	0.6514	0.5138	0.477	2.196	0.6779	0.6431	0.4634	0.495	2.231
48	0.6888	0.6855	0.5370	0.457	2.088	0.6855	0.6651	0.4849	0.495	2.232
49	0.6474	0.5373	0.3895	0.412	1.892	0.6909	0.6720	0.4954	0.500	2.219
50	0.7047	0.6535	0.5168	0.475	2.144	0.6781	0.6400	0.4585	0.494	2.212
51	0.6785	0.6200	0.4719	0.446	2.082	0.6898	0.6706	0.4940	0.494	2.235
Mean	0.666	0.615	0.458	0.447	2.050	0.684	0.658	0.477	0.493	2.228
Std	0.028	0.046	0.048	—	—	0.005	0.012	0.014	—	—

The per-seed values make the variance pattern concrete. On aPY-15 the K-means NMI spans 0.623 to 0.705 across seeds while the consensus spans 0.677 to 0.691. On the full aPY set, however, the consensus ACC and ARI are visibly bimodal—seeds 43, 44, 46, 50 and 51 land near 0.61 ACC and seeds 42, 45, 47, 48 and 49 near 0.55—which is the source of the elevated ACC and ARI standard deviations discussed in Section 4.4.2.

A.2. Early Experiment Results

This section documents the results from approaches attempted before the final DDECCS pipeline, providing context for the design decisions discussed in Chapter 6.

A.2.1 Autoencoder-Based Approach

The initial approach used convolutional autoencoders with tag-prediction branches, trained with reconstruction + tag BCE + consensus losses. These variants—with and without the tag-prediction branch, at 128×128 and 224×224 resolution, and with a frozen ResNet-101 backbone—all produced low-quality clusterings far below the frozen-features K-means baseline (NMI = 0.869 on AwA2). This is what motivated abandoning the autoencoder design in favor of pretrained features.

A later line-by-line review of the archived training code found that these runs were compromised beyond the design issues discussed in Chapter 6. Most importantly, the consensus term contributed no gradient at all: the embeddings it compared were computed inside a `torch.no_grad()` block, so the consensus loss was recorded in the logs but could not influence the weights, and the models were in effect trained on reconstruction and tag prediction alone. Secondary defects compounded this—the consensus target was rebuilt only once every five epochs, the optimizer state was re-initialized each epoch, and the co-association target mixed value ranges—so the reported autoencoder results measure a partially specified objective rather than the intended one. They therefore show that *these* training runs failed; they do not establish that consensus-guided representation learning fails on this data.

A.2.2 DDC-Style MLP Training

The second approach used DDC's actual loss functions (MI + pairwise) with a trainable 3-layer MLP on frozen ResNet features.

Table A.4: MLP training results on AwA2. Per-instance tag masking ($r = 0.5$) was needed for the self-generated pairwise constraints to be meaningful; even so, results did not scale from the 1,600-sample subset to the full dataset.

Configuration	**N**	**Epochs**	**NMI**	**Active Clusters**
No tag masking (sample)	1,600	10	0.052	13/50
With tag masking $r = 0.5$ (sample)	1,600	10	0.056	13/50
With tag masking $r = 0.5$ (sample)	1,600	200	0.124	30/50
Full dataset, $\lambda_P = 1.0$	29857	200	0.006	28/50
Full dataset, $\lambda_P = 6.0$	29857	200	0.000	1/50
Full dataset, lr $= 10^{-4}$	29857	200	0.006	24/50
K-means on raw features	29857	—	0.869	50/50

As the final row shows, K-means on raw features outperformed every MLP attempt, which led to the frozen-features pipeline.

The same review identified two implementation gaps in this variant. First, DDC's defining mechanism was never active: the ILP that selects informative tags was fully implemented but never invoked during training, so the pairwise constraints were always generated from the raw tag space rather than from the ILP-filtered one, leaving the model permanently in what DDC treats as a pre-training phase. Second, the constraint generation itself deviated from DDC: the class-level continuous predicates make the tag-distance term dominate DDC's pair-selection score by several orders of magnitude, disabling the intended preference for tag-similar but cluster-different pairs, and the per-batch random tag masking (re-drawn each step) replaces DDC's one-time missing-tag simulation with mean imputation, making partner selection between same-class instances depend on mask sampling rather than semantics. The early cluster collapse described in Section 6.2—the MI objective committing the model to roughly 24 coarse clusters within one epoch—is confirmed and remains the proximate optimization failure; the gaps above explain why the pairwise term had no corrective signal to offer.

A.3. ILP Solver Configuration

The ILP is solved using PuLP's CBC solver with the following configuration:

Table A.5: ILP solver parameters and behavior across datasets. Tag counts and tags-per-cluster are for the representative run (seed = 42) shown in Appendix B; average TC/ITF in Chapter 4 are over all 10 runs.

Parameter	**AwA2**	**aPY (32)**	**aPY (15)**
Attribute density	0.217	0.111	0.087
α (adaptive)	3	2	2
β (solved)	3	2	1
Tags selected	73/85	52/64	43/64
Tags per cluster	4.7	5.1	6.0
Solve time	$\sim$60s	$\sim$30s	$\sim$1s

These solutions were obtained under a 30-second limit per β, and the solver's termination status was not inspected, so a time-limited incumbent could not be distinguished from a proven optimum. All six seed-42 instances were subsequently re-solved at extended limits: four are proven optimal at the β reported here—both description modes on aPY-15, `ddeccs` on aPY-32 and `ddeccs` on AwA2—while `ddc` on aPY-32 and `ddc` on AwA2 are time-limited incumbents. This thesis reports the smallest *certified* β, the smallest value at which a solution has been proven optimal; under that convention the values above stand. Appendix C.5 gives the audit in full.

B. Cluster Descriptions and Dataset Details

This appendix lists the complete per-cluster ILP descriptions for both description modes on all three datasets, followed by the dataset statistics, the ensemble configuration, and the source-code layout.

B.1. Complete ILP Cluster Descriptions

This section gives the full per-cluster descriptions summarized in Chapter 4. Sections B.1.1–B.1.3 give the `ddc` mode (ILP applied to the K-means clustering) and Sections B.1.4–B.1.6 the `ddeccs` mode (ILP applied to the consensus clustering), both at the representative seed.

B.1.1 AwA2 Cluster Descriptions (ddc mode, K = 50)

Table B.1 presents the complete ILP-generated descriptions for all 50 AwA2 clusters. The ILP solved with $\alpha = 3$, $\beta = 3$, selecting 73 of 85 attributes.

The descriptions below are from a representative run (seed = 42). Because both the clustering and the ILP are recomputed per seed, the specific attributes and tag counts vary slightly across runs, whereas the average TC and ITF reported in Chapter 4 are over all 10 runs.

Table B.1: Complete ILP cluster descriptions for all 50 AwA2 clusters (ddc mode, seed=42).

C	ILP Description	n	TC	ITF
0	strong, fish, arctic, fierce	692	0.76	2.77
1	bulbous, chewteeth, grazer, plains, fields, ground	499	0.50	2.51
2	smelly, muscle, bipedal, jungle, tree	658	0.61	3.14
3	stripes, chewteeth, plains, group	941	0.76	2.61
4	hooves, longleg, horns, plains, timid	1050	0.61	2.44
5	gray, swims, skimmer, water, timid	799	0.62	2.82
6	hairless, toughskin, bulbous, horns, strong	523	0.60	2.48
7	black, big, horns, plains, ground	701	0.60	2.39
8	hooves, longleg, fast, fields	1037	0.77	2.60
9	flippers, swims, arctic, ocean	891	0.78	2.84
10	slow, strong, quadrapedal, inactive, grazer	368	0.63	2.54
11	tail, meatteeth, walks, meat, newworld	421	0.61	2.48
12	patches, big, meatteeth, active, meat	599	0.62	2.54
13	orange, stripes, muscle, fierce	697	0.82	3.12
14	meatteeth, claws, fast, hunter	488	0.75	2.47
15	furry, hops, vegetation, ground	874	0.77	2.82
16	furry, walks, fast, agility, newworld, ground	806	0.51	2.41
17	bush, jungle, tree, smart	617	0.78	3.19
18	brown, big, quadrapedal, newworld	471	0.75	2.47
19	brown, big, quadrapedal, grazer, fields	1052	0.62	2.58
20	gray, toughskin, slow, oldworld	388	0.76	2.54
21	white, quadrapedal, vegetation, group	299	0.75	2.60
22	black, lean, claws, agility, newworld	315	0.60	2.35
23	gray, bipedal, active, agility, tree	938	0.61	2.63
24	meatteeth, claws, walks, agility, solitary, domestic	271	0.52	2.57
25	black, small, claws, flys, forest, cave	490	0.51	2.91
26	active, meat, hunter, stalker	465	0.75	2.47
27	small, flippers, active, fish, water	606	0.61	2.54
28	black, spots, lean, paws	435	0.75	2.54
29	hooves, longleg, horns, forest, timid	1039	0.63	2.48
30	white, walks, weak, timid, group	311	0.66	2.68
31	brown, lean, hunter, stalker, smart	474	0.61	2.58
32	patches, bulbous, slow, oldworld	684	0.75	2.56
33	toughskin, bulbous, hooves, smelly, inactive, newworld	514	0.50	2.61
34	gray, hairless, tusks, oldworld	481	0.76	2.94
35	spots, longleg, longneck, plains	920	0.77	2.89
36	meat, hunter, stalker, fierce	810	0.77	2.53
37	white, vegetation, grazer, group	530	0.75	2.60
38	strong, hibernate, forest, fierce	650	0.76	2.87
39	hairless, toughskin, bulbous, slow, water	555	0.61	2.44
40	furry, paws, tail, domestic	482	0.75	2.53
41	black, patches, stripes, nocturnal, forest, smart	456	0.50	2.86
42	small, chewteeth, buckteeth, tunnels, vegetation, domestic	799	0.51	2.92
43	blue, flippers, fast, fish, ocean	356	0.61	2.92
44	pads, paws, claws, slow, inactive, timid	489	0.51	2.69
45	brown, tail, buckteeth, swims	192	0.76	2.77
46	spots, lean, agility, stalker	577	0.75	2.54
47	hairless, flippers, ocean, water	561	0.77	2.60
48	small, weak, active, ground, solitary	404	0.64	2.71
49	furry, paws, tail, domestic	182	0.75	2.53

B.1.2 aPY 15-Class Cluster Descriptions (ddc mode, K = 15)

Table B.2 presents the complete ILP descriptions for the 15-class aPY subset. The ILP solved with $\alpha = 2$, $\beta = 1$, selecting 43 of 64 attributes.

Table B.2: Complete ILP cluster descriptions for aPY 15-class subset.

C	ILP Description	n	TC	ITF
0	Round, Vert Cyl, Occluded, Label, Furn. Leg, Wood, Clear	235	0.29	1.68
1	Wheel, Door, Taillight, Side mirror, Exhaust	435	0.41	2.07
2	2D Boxy, Round, Plastic, Glass, Clear, Shiny	169	0.34	1.54
3	Head, Eye, Torso	860	0.67	2.25
4	Furn. Back, Furn. Seat, Furn. Arm, Leather	123	0.51	2.01
5	Vert Cyl, Label, Furn. Leg, Furn. Back, Metal, Plastic, Glass, Clear	440	0.25	1.69
6	2D Boxy, 3D Boxy, Furn. Leg, Furn. Arm, Cloth, Shiny	544	0.35	1.68
7	2D Boxy, Round, Horiz Cyl, Text, Rein, Metal, Plastic, Cloth, Furry, Wool, Leather	251	0.18	1.82
8	3D Boxy, Horiz Cyl, Window, Row Wind, Text	221	0.40	1.71
9	Leg, Foot/Shoe, Furry	430	0.70	2.48
10	2D Boxy, Occluded, Window, Furn. Leg, Furn. Seat, Wood, Shiny	225	0.29	1.61
11	2D Boxy, Horiz Cyl, Row Wind, Wheel, Mast, Text, Metal, Wood	307	0.25	1.70
12	Tail, Ear, Snout	280	0.69	2.71
13	Head, Torso, Window, Wheel, Door, Headlight, Taillight, Side mirror, Label, Feather	446	0.20	2.03
14	Pedal, Handlebars, Text, Plastic	223	0.53	2.01

B.1.3 aPY Full Cluster Descriptions (ddc mode, K = 32)

Table B.3 presents the complete ILP descriptions for all 32 clusters of the full aPY dataset. The ILP solved with $\alpha = 2$, $\beta = 2$, selecting 52 of 64 attributes.

Table B.3: Complete ILP cluster descriptions for aPY full dataset (ddc mode, 32 clusters, seed=42).

C	ILP Description	n	TC	ITF
0	Window, Row Wind, Wood, Glass	228	0.52	1.95
1	Foot/Shoe, Rein, Saddle, Furry	336	0.51	2.62
2	Mouth, Hand, Arm, Foot/Shoe	508	0.50	2.10
3	Furn. Leg, Furn. Back, Furn. Seat, Wood	551	0.53	1.99
4	Tail, Ear, Snout, Torso, Leg, Metal, Furry, Wool	359	0.26	2.26
5	Beak, Wing, Feather	290	0.71	2.64
6	2D Boxy, 3D Boxy, Window, Row Wind, Wheel, Door, Plastic, Wood, Glass	383	0.23	2.05
7	3D Boxy, Door, Headlight, Side mirror	300	0.50	2.77
8	Ear, Foot/Shoe, Metal, Shiny	160	0.51	1.82
9	2D Boxy, Hair, Foot/Shoe, Furn. Leg, Furn. Back, Furn. Seat, Wood, Cloth	233	0.25	1.98
10	Mouth, Face, Eye, Hand, Metal, Shiny	391	0.34	1.89
11	Nose, Mouth, Hair, Hand, Furn. Leg, Furn. Back	674	0.34	2.04
12	Vert Cyl, Occluded, Label, Furn. Back, Plastic, Glass, Shiny	586	0.29	2.17
13	Horiz Cyl, Wing, Jet engine	219	0.69	2.87
14	Occluded, Head, Window, Metal, Cloth, Shiny	397	0.34	1.87
15	3D Boxy, Window, Wheel, Headlight	536	0.51	2.15
16	Hair, Arm, Skin	346	0.67	2.27
17	Wheel, Handlebars, Plastic, Shiny	285	0.53	1.96
18	Head, Eye, Hand, Leg, Metal	342	0.44	2.03
19	2D Boxy, Face, Torso, Furn. Leg, Wood, Cloth	230	0.34	1.97
20	Tail, Snout, Leg	153	0.74	2.10
21	Occluded, Tail, Beak, Eye, Wing, Window, Row Wind, Saddle, Metal, Cloth, Feather	748	0.18	2.21
22	Tail, Ear, Furry	306	0.68	2.20
23	2D Boxy, 3D Boxy, Vert Cyl, Screen, Plastic, Glass, Shiny	786	0.30	2.17
24	Horiz Cyl, Handlebars, Text, Plastic	411	0.54	2.62
25	Face, Arm, Skin	266	0.68	2.27
26	Leaf, Pot, Vegetation	222	0.83	3.47
27	Occluded, Nose, Mouth, Foot/Shoe	375	0.50	2.02
28	Vert Cyl, Nose, Hair, Face, Torso, Leg, Window, Skin	282	0.26	2.10
29	2D Boxy, Nose, Eye, Torso, Furn. Back, Furn. Seat, Furn. Arm, Wood	216	0.27	2.18
30	Tail, Head, Snout	750	0.67	2.20
31	Wheel, Pedal, Handlebars, Metal, Shiny	402	0.43	2.19

B.1.4 AwA2 Cluster Descriptions (ddeccs mode, K = 50)

Table B.4 gives the `ddeccs`-mode descriptions for all 50 AwA2 clusters, the consensus-clustering counterpart to Table B.1.

Table B.4: Complete ILP cluster descriptions for all 50 AwA2 clusters (`ddeccs` mode).

C	ILP Description	n	TC	ITF
0	black, white, spots, lean	438	0.76	2.54
1	hooves, horns, vegetation, grazer, timid	2412	0.62	2.77
2	black, patches, slow, oldworld	689	0.76	3.05
3	furry, small, quadrapedal, vegetation, newworld	1286	0.60	2.44
4	spots, longleg, longneck, grazer	947	0.77	2.89
5	white, hooves, chewteeth, group	956	0.76	2.87
6	tail, strong, vegetation, grazer, domestic	1717	0.61	2.44
7	meatteeth, walks, quadrapedal, meat	730	0.76	2.53
8	flippers, swims, arctic, ocean	888	0.77	2.84
9	hooves, longleg, fast, fields	1273	0.76	2.47
10	paws, claws, walks, newworld, oldworld	986	0.61	2.54
11	white, quadrapedal, fields, group	1166	0.75	2.47
12	black, stripes, active, nocturnal, forest	396	0.62	2.82
13	small, walks, ground, domestic	383	0.76	2.53
14	meatteeth, meat, hunter, fierce	832	0.76	2.53
15	white, paws, newworld, ground, domestic	497	0.61	2.48
16	paws, fast, active, solitary, domestic	24	0.61	2.58
17	orange, stripes, claws, muscle	703	0.80	3.09
18	gray, hairless, toughskin, bulbous, horns	538	0.60	2.64
19	gray, tail, forager, tree	918	0.76	2.70
20	lean, flippers, swims, fish, water	663	0.62	2.60
21	hands, bipedal, jungle, group	571	0.77	2.87
22	gray, toughskin, bulbous, slow, inactive	560	0.65	3.00
23	hands, longleg, agility, jungle, tree	257	0.61	2.66
24	big, bulbous, hooves, smelly, vegetation, newworld	516	0.51	2.61
25	black, stripes, smelly, fields	147	0.81	2.72
26	strong, fish, arctic, fierce	700	0.76	2.77
27	claws, strong, hibernate, forest	652	0.77	2.61
28	hairless, flippers, ocean, water	804	0.76	2.60
29	plankton, ocean, water, smart	710	0.75	2.94
30	furry, hops, vegetation, ground	891	0.76	2.82
31	orange, agility, stalker, forest, smart	414	0.60	2.54
32	active, meat, stalker, fierce	484	0.75	2.53
33	gray, toughskin, tusks, oldworld	514	0.78	2.70
34	hairless, flippers, swims, fish	238	0.76	2.67
35	furry, bipedal, oldworld, jungle	686	0.75	2.70
36	meatteeth, fast, quadrapedal, hunter	494	0.75	2.53
37	tail, active, agility, tree	34	0.76	2.53
38	paws, walks, meat, newworld, ground	178	0.60	2.48
39	hairless, toughskin, big, tusks	364	0.75	2.70
40	lean, fast, stalker, jungle	141	0.77	2.47
41	brown, small, lean, forest, fields, fierce	169	0.51	2.41
42	orange, hunter, forest, smart, solitary	39	0.60	2.60
43	small, buckteeth, hibernate, forager, tree	57	0.60	2.86
44	spots, lean, agility, stalker	428	0.76	2.54
45	brown, tail, buckteeth, water	159	0.77	2.70
46	brown, meatteeth, hunter, smart, solitary	46	0.60	2.58
47	furry, big, strong, hibernate	33	0.76	2.60
48	brown, big, grazer, plains, group	1090	0.60	2.62
49	longleg, horns, grazer, plains, fields	39	0.61	2.63

B.1.5 aPY 15-Class Cluster Descriptions (ddeccs mode, K = 15)

Table B.5 gives the `ddeccs`-mode descriptions for the 15-class aPY subset, the consensus-clustering counterpart to Table B.2.

Table B.5: Complete ILP cluster descriptions for aPY 15-class subset (`ddeccs` mode).

C	ILP Description	n	TC	ITF
0	Ear, Eye, Leg, Foot/Shoe	1159	0.53	2.53
1	Vert Cyl, Label, Clear, Shiny	352	0.52	2.01
2	2D Boxy, Horiz Cyl, Row Wind, Sail, Mast, Plastic, Wood	249	0.29	2.06
3	Furn. Leg, Furn. Back, Furn. Seat, Cloth	928	0.51	2.53
4	3D Boxy, Occluded, Wheel, Door, Headlight, Taillight, Side mirror, Label	507	0.26	2.10
5	Pedal, Handlebars, Text, Plastic	229	0.52	2.01
6	Wheel, Door, Headlight, Side mirror, Exhaust	578	0.41	2.15
7	Tail, Snout, Furry	258	0.70	2.71
8	3D Boxy, Horiz Cyl, Window, Row Wind, Text	149	0.40	1.88
9	Occluded, Window, Text, Metal, Wood	85	0.42	1.88
10	Head, Torso, Wing	33	0.68	2.71
11	2D Boxy, Round, Vert Cyl, Text, Rein, Metal, Plastic, Cloth, Leather	247	0.22	1.97
12	2D Boxy, Round, Plastic, Glass, Clear, Shiny	166	0.34	1.95
13	Hair, Face, Arm	125	0.87	2.71
14	Beak, Foot/Shoe, Feather	124	0.71	2.48

B.1.6 aPY Full Cluster Descriptions (ddeccs mode, K = 32)

Table B.6 gives the `ddeccs`-mode descriptions for all 32 clusters of the full aPY dataset, the consensus-clustering counterpart to Table B.3.

Table B.6: Complete ILP cluster descriptions for aPY full dataset (`ddeccs` mode, 32 clusters).

C	ILP Description	n	TC	ITF
0	2D Boxy, 3D Boxy, Round, Vert Cyl, Text, Rein, Cloth, Furry, Leather	248	0.22	2.66
1	Torso, Handlebars, Skin, Metal, Cloth	534	0.40	2.14
2	Tail, Ear, Furry	299	0.68	2.27
3	Torso, Arm, Cloth	253	0.67	2.18
4	Leaf, Pot, Vegetation	188	0.85	3.47
5	Occluded, Furn. Leg, Furn. Back, Furn. Seat	731	0.52	3.12
6	Snout, Leg, Foot/Shoe	460	0.68	2.41
7	Hair, Face, Arm, Skin	259	0.51	2.22
8	Head, Hair, Face, Skin	2174	0.51	2.22
9	Vert Cyl, Label, Glass	298	0.67	2.73
10	Tail, Beak, Feather	270	0.77	3.00
11	Occluded, Head, Hair, Torso, Skin	1378	0.40	2.14
12	Horiz Cyl, Wing, Propeller, Jet engine	277	0.55	3.12
13	Tail, Mouth, Hand	137	0.69	2.77
14	3D Boxy, Window, Row Wind, Wood	243	0.51	2.17
15	Screen, Plastic, Glass	244	0.71	2.73
16	Window, Row Wind, Door, Shiny	93	0.52	2.17
17	Horiz Cyl, Handlebars, Text, Plastic, Cloth	551	0.40	2.33
18	2D Boxy, 3D Boxy, Occluded, Metal, Wood	189	0.40	2.03
19	Head, Hair, Arm	1022	0.67	2.27
20	Nose, Mouth, Face	135	0.83	2.87
21	Vert Cyl, Glass, Shiny	172	0.71	2.27
22	Row Wind, Door, Headlight, Side mirror	73	0.54	2.27
23	Tail, Leg, Furry	159	0.74	2.27
24	Wheel, Door, Headlight, Shiny	600	0.51	2.10
25	2D Boxy, 3D Boxy, Horiz Cyl, Occluded, Wheel, Door, Metal, Plastic, Wood, Shiny	132	0.20	2.09
26	Wheel, Headlight, Side mirror, Handlebars	235	0.50	2.40
27	Horiz Cyl, Window, Door, Headlight	26	0.50	2.10
28	Snout, Eye, Foot/Shoe	347	0.67	2.41
29	Ear, Eye, Torso, Leg, Foot/Shoe	238	0.40	2.25
30	2D Boxy, 3D Boxy, Mast, Metal, Wood	27	0.41	2.31
31	Ear, Eye, Foot/Shoe	279	0.69	2.27

B.2. Dataset Statistics

This section summarizes the key statistics of the two datasets used in our evaluation.

B.2.1 AwA2 Dataset

Table B.7 summarizes the AwA2 dataset used for evaluation.

Table B.7: AwA2 dataset statistics. AwA2 [Xian et al., 2019] replaces the original AwA dataset [Lampert et al., 2014], which was withdrawn due to copyright restrictions. Both datasets share the same 50 classes and 85 attributes.

Property	**Value**
Total images	37,322
Training split (80%)	29857
Number of classes	50
Number of attributes	85
Attribute type	Continuous, per-class
Attribute density	≈ 0.217
Image source	Flickr, Wikipedia

B.2.2 aPY Dataset

Table B.8 summarizes the aPY dataset and its 15-class subset.

Table B.8: aPY dataset statistics. The dataset combines Pascal VOC 2008 and Yahoo images [Farhadi et al., 2009].

Property	**Value**
Total object instances	15,339
Training split (80%)	12,271
Number of classes (full)	32
Number of classes (DDC subset)	15
DDC subset training instances	5,189
Number of attributes	64
Attribute type	Per-instance binary, aggregated to per-class means
Attribute density	≈ 0.111
Annotation type	Per-object bounding box crop

The 15 classes in the DDC subset are: dog, cat, monkey, zebra, bird, bag, mug, bottle, chair, car, building, bicycle, boat, sofa, and dining table. These were identified from the ontology visualization in DDC's Figure 3.

B.3. DECCS Ensemble Details

The DECCS consensus step combines four base clustering algorithms; Table B.9 lists each algorithm and its configuration.

Table B.9: DECCS base clustering algorithms and their configurations.

Algorithm	**Library**	**Configuration**
K-means (ensemble member)	scikit-learn	Elkan algorithm, n_init = auto, $max\_iter = 300$
Spectral	scikit-learn	Nearest-neighbor affinity ($k = 15$, scikit default), arpack solver
GMM	scikit-learn	Full covariance, $reg\_covar = 10^{-3}$, PCA to $\min(256, \dim, \max(64, N/2K))$ dims
Agglomerative	scikit-learn	Ward linkage
Consensus	scikit-learn	Sparse co-association matrix via k-NN ($k = 20$), final Spectral Clustering

B.4. Implementation Details

Table B.10 lists the main source files and their purposes.

Table B.10: Source code files and their roles in the DDECCS pipeline.

File	**Purpose**
`main_experiments.py`	Main pipeline: feature extraction, clustering, ILP descriptions, evaluation
`dataset.py`	`AttributeDataset` class, dataset config registry, aPY 15-class filter
`utils.py`	DECCS ensemble, consensus matrix, evaluation metrics, I/O utilities
`visualize.py`	t-SNE and PCA visualization, cluster sample saving
`setup_apy.py`	aPY dataset preprocessing (annotation parsing, bounding box cropping)
`run_all.sh`	Experiment orchestration script (all datasets, all modes, 10 seeds)

C. Reproduction Details

This appendix supports Chapter 5. It gives the per-configuration results underlying the chapter's tables, the specification extracted from the published paper, the judgment calls made where that specification is silent, the recovery and verification of the aPY per-instance annotations, the solver certification audit, the defect inventory found across both codebases, the runtimes, and the code layout. Nothing here changes any figure reported in Chapter 5; the material is separated out so the chapter can state the findings without carrying their supporting detail.

C.1. Per-Configuration Results

Tables C.1 and C.2 give every configuration run for Chapter 5, including those omitted from the chapter's tables. All values are final-epoch, over five seeds (42–46). Best-epoch selection was not used anywhere: choosing the epoch by NMI requires the ground-truth labels and inflated results by 0.006 on average on AwA2 and 0.033 on aPY-15, and by as much as 0.086 in a single run, so it is reported nowhere in this thesis. The individual per-run artefacts — `history.csv`, `summary.json`, `ilp_descriptions.json` and the cluster assignments — are written to `results_v2/ddc_phase1/<dataset>/<tag>/` under the run tag given in each table.

Table C.1: AwA2 reproduction, all configurations ($N = 29{,}857$, $K = 50$, $\alpha = 3$, raw features, $r = 0.5$, $\gamma = 100$, $l = 1$). Five seeds (42–46), final epoch, mean ± std.

Configuration	Run tag	NMI	ACC	ARI	Acti
$\mu = 1.0$, $\lambda = 1$, ILP on	`full_mu10_a3_ilp`	0.8081 ± 0.0237	0.4861 ± 0.0454	0.4480 ± 0.0505	31.0 ±
$\mu = 1.5$, $\lambda = 1$, ILP on	`full_mu15_a3_ilp`	0.9229 ± 0.0062	0.8306 ± 0.0259	0.8276 ± 0.0251	49.4 ±
$\mu = 1.5$, $\lambda = 0$	`full_mu15_lam0_ilp`	0.7425 ± 0.0186	0.6596 ± 0.0318	0.5639 ± 0.0392	37.4 ±
$\mu = 1.5$, $\lambda = 1$, ILP off	`full_mu15_a3_noilp`	0.9255 ± 0.0060	0.8444 ± 0.0222	0.8413 ± 0.0241	50.0 ±
$\mu = 1.5$, ILP at 120 s	`full_mu15_a3_ilp_t120`	0.9252 ± 0.0032	0.8446 ± 0.0135	0.8417 ± 0.0145	49.8 ±
Matched k-means baselines (5 seeds, `n_init` = 10)					
Features only	—	0.8699 ± 0.0021	0.7864 ± 0.0099	0.7469 ± 0.0098	50/5
Features ⊕ raw tags	—	0.8804 ± 0.0028	0.8033 ± 0.0081	0.7693 ± 0.0101	50/5
Features ⊕ standardized tags	—	0.7665 ± 0.0079	0.5194	0.3767	50/5
Tags only (control)	—	0.5909 ± 0.0000	0.4976 ± 0.0000	0.0439	50/5

The same-class pair fraction is 0.9215 ± 0.0021 in every AwA2 configuration above and is therefore omitted from the table. The features-only baseline reproduces the corresponding figure of Chapter 4 (0.8699 against 0.869), which confirms that the split, the feature cache and the tag alignment used by the reproduction harness match those of the main pipeline.

Table C.2: aPY-15 reproduction, all configurations ($N = 5{,}189$, $K = 15$, per-instance binary tags, $r = 0.5$). Five seeds (42–46), final epoch, mean ± std. TC and ITF at $\alpha = 3$ are over the two seeds for which the ILP was feasible on the final assignment.

Configuration	Run tag	NMI	ACC	ARI	Active	Same-
$\mu = 1.0$, $\alpha = 1$	`apy15_mu10_a1_ilp`	0.6337 ± 0.0153	0.5630 ± 0.0251	0.4731 ± 0.0315	11.2 ± 1.9	0.52
$\mu = 1.5$, $\alpha = 1$	`apy15_mu15_a1_ilp`	0.6368 ± 0.0108	0.5610 ± 0.0200	0.4785 ± 0.0314	15.0	0.55
$\mu = 1.5$, $\alpha = 2$	`apy15_mu15_a2_ilp`	0.6735 ± 0.0091	0.5974 ± 0.0362	0.5082 ± 0.0358	15.0	0.62
$\mu = 1.5$, $\alpha = 3$	`apy15_mu15_a3_ilp`	0.6710 ± 0.0045	0.5986 ± 0.0211	0.5120 ± 0.0338	15.0	0.62
$\mu = 1.5$, $\alpha = 4$	`apy15_mu15_a4_ilp`	0.6768 ± 0.0056	0.6199 ± 0.0160	0.5263 ± 0.0245	15.0	0.62
$\mu = 1.5$, $\lambda = 0$	`apy15_mu15_a1_lam0`	0.5318 ± 0.0216	0.4751 ± 0.0269	0.3215 ± 0.0235	15.0	0.54
$\mu = 1.5$, ILP off	`apy15_mu15_a1_noilp`	0.6798 ± 0.0090	0.6251 ± 0.0263	0.5284 ± 0.0265	15.0	0.62
Matched k-means baselines (5 seeds, `n_init` = 10)						
Features only	—	0.6463 ± 0.0196	0.6012	0.4298	15/15	—
Features ⊕ tags	—	0.6767 ± 0.0182	0.6166	0.4700	15/15	—
Tags only (control)	—	0.3778 ± 0.0026	0.3239	0.0544	15/15	—

The $\alpha = 4$ row is an internal control rather than an operating point: the ILP is infeasible in all 20 rounds, so g is left as the identity and no descriptions are produced. On four of the five seeds these runs are bit-identical to the corresponding ILP-off runs, which confirms that the differences reported in Table 5.3 are caused by the mask and not by any other difference between the two code paths.

C.2. The Specification as Extracted from the Paper

Table C.3 lists each element of the original DDC as implemented, with the location in the published paper from which it was taken. Equation numbers refer to the paper's own numbering; where this thesis restates the same formulation, the corresponding equation in Chapter 3 is given.

Table C.3: The original DDC as implemented, with the source of each element in the published paper [Zhang and Davidson, 2021].

Element	Source	As implemented
Backbone	Sec. 4.1	Frozen ResNet-101, 2048-d, no fine-tuning
Encoder	Sec. 3.1	Three fully connected layers, softmax over K
Clustering objective	Eq. (1)	Mutual information between inputs and assignments
Symbolic objective	Eq. (2)–(4)	ILP as in Eq. 3.1–3.3
Tag mask g	Sec. 3.2	Derived from the ILP allocation W each round
Pair selection	Eq. (5)	Masked tag distance plus cluster disagreement, $\gamma = 100$
Partners per anchor	Sec. 3.3	$l = 1$
Pairwise loss weight	Sec. 3.3	$\lambda = 1$
Coverage parameter	Sec. 4.1	$\alpha = 8$ global (see Sec. 5.6.1)
Orthogonality bound	Sec. 4.1	Smallest feasible β, searched upward from 1
Annotation ratio	Sec. 4.2	$r = 0.5$ for the reported configuration
Description metrics	Eq. (8)–(9)	TC and ITF as in Eq. 4.1–4.2

C.3. Judgment Calls

The paper does not state every setting needed to run the method. Each gap below was resolved once, before any results were collected, and applied identically to every configuration in Chapter 5.

Optimizer and learning rate. Adam [Kingma and Ba, 2015] with default parameters, as the paper states, at learning rate 10^{-3}. The paper gives no value; 10^{-3} is Adam's own default and the only choice that does not amount to tuning against the result.

Batch size. 256, matching the archived earlier attempt of Section 4.7 so that the two are directly comparable.

Epoch budget. The paper trains "until convergence" without stating a budget. We fixed 10 pretraining epochs followed by 20 rounds of 2 epochs each. A fixed budget removes convergence criteria as a source of difference between configurations, at the cost of not guaranteeing that any single configuration has converged.

Preprocessing. None: features enter raw. Section 4.4.4 establishes that standardization costs NMI on these features, and the reproduction is consistent with the rest of the thesis in this respect.

Norm and output in Equation (5). $|\cdot|$ taken as L_2; $f_\theta(x)$ taken as the softmax output rather than a logit or an intermediate embedding.

Interpretation of r. Read as a ratio of annotated *instances*, with the mask drawn once and held fixed for the whole run (`--mask_mode instance`). The alternative reading, in which individual entries are masked and re-drawn, is implemented behind `--mask_mode entry` and available for ablation. The instance-level reading is the one consistent with the paper's description of simulating incomplete annotation, and redrawing per batch was one of the defects identified in the earlier attempt (Appendix A.2).

Pair-partner restriction. Partners are drawn only from annotated instances (`--pairs_from annotated`). Under instance-level masking every unannotated instance carries the identical imputed tag vector, so admitting them makes the tag-distance term of Equation (5) uninformative for a large fraction of candidate pairs.

Tag representation. On AwA2, binary tags from the dataset's `predicatematrix-binary.txt`, with a midpoint threshold of the continuous matrix as fallback where a binary matrix is not distributed. On aPY, the per-instance annotations recovered in Section C.4. Both differ from the DDECCS pipeline of Chapter 3, which uses the continuous per-class matrix throughout: on AwA2 the same attributes are binarised differently (densities 0.365 and 0.217), whereas on aPY the tags are different objects entirely, per-instance rather than per-class means, at effectively the same mean density (0.0869 against 0.087). TC and ITF are therefore not comparable across the two chapters on either dataset, even at equal α.

ITF base. Reported in nats throughout this thesis. The paper's table is base-2; conversion is a factor of $\ln 2$.

ILP warm start. The search over β starts from last $\beta - 1$ with a full search as fallback. This is an optimization only and does not change the solution returned.

The one deviation. A weight μ on the marginal entropy term $H(Y)$ of Equation (1), which the published equation does not carry. Implemented as written, at $\mu = 1.0$, the objective leaves 15 to 24 of the 50 AwA2 clusters unpopulated. Both settings are reported together throughout Chapter 5, with $\mu = 1.0$ as the reproduction result and $\mu = 1.5$ identified as a deviation wherever it appears.

C.4. Recovery of the aPY Per-Instance Annotations

Chapter 4 notes that the preprocessing used throughout this thesis averages aPY's per-instance binary annotations into per-class means, so that both datasets enter the pipeline in the same form. Testing the constraint mechanism of Section 5.4 required the annotations at instance level, since averaging them reintroduces exactly the per-class degeneracy the test is meant to avoid. They were therefore reconstructed from the original annotation files by `build_apy_instance_tags.py`, which writes a sidecar at `data/aPY-data/aPY/per-instance-attributes.tsv` and leaves the existing preprocessing untouched.

Because a misaligned reconstruction would silently produce plausible but wrong results, the mapping was verified at three levels. First, the object indices parsed from `labels.txt` must be exactly $0 \ldots N-1$, which fails immediately on any gap or duplicate. Second, every record must match an annotation entry, in order, on both class name and source image stem. Third, the per-class means of the recovered vectors must reproduce `predicate-matrix-continuous.txt`, an independent artefact written by the original preprocessing, so that a single misassigned row breaks the check.

The third tier was tested against deliberate corruptions before use. A one-instance shift is caught; a cross-class misassignment is caught; a permutation of two instances within a single class is *not*, because class means are invariant to it. That residual case can only arise between two objects of the same class in the same source image, and it does not affect any per-class or per-cluster quantity reported in this thesis. A synthetic pre-deployment test over 752 constructed instances passed at a maximum absolute deviation of 4.88×10^{-7}.

On the real data, all 15,339 annotation entries matched all 15,339 processed instances with zero skipped, at a maximum absolute deviation of 4.98×10^{-7} against the reference matrix. The recovered mapping is therefore the identity permutation. The resulting matrix has a per-instance binary density of 0.0869, or 5.56 active attributes per instance on average, ranging from 0 to 16; it contains 1,703 distinct rows before masking and between 1,080 and 1,107 at $r = 0.5$, against 51 for AwA2.

C.5. Solver Certification Audit

The ILP search reports the smallest β at which a solution is found. Whether that value is meaningful depends on the solver correctly distinguishing a proven optimum from a time-limited incumbent, and on a timeout without an incumbent not being mistaken for infeasibility. Both distinctions were found to be mishandled (Section C.6), so the six shipped ILP solutions underlying Chapter 4 were re-solved at extended time limits to establish what they certify.

All six are seed-42 solutions, covering both description modes on each of the three dataset configurations. Four are proven optimal at the β reported: both modes on aPY-15, `ddeccs` on aPY-32, and `ddeccs` on AwA2. Two are time-limited incumbents rather than optima: `ddc` on aPY-32 reports objective 164 where 151 is achievable, and `ddc` on AwA2 reports 234 where 224 is achievable. On AwA2, $\beta = 2$ was shown to be feasible in both modes, but only after 900 seconds, and both incumbents sit roughly 117% above their linear-programming bounds, so the true $\beta = 2$ optima are unknown.

The convention adopted for this thesis is the smallest *certified* β: the smallest value at which a solution has been proven optimal. Under that convention the β values

reported in Chapter 4 and Appendix B stand, and no figure in those tables changes. Reporting the uncertified $\beta = 2$ solutions instead would substitute one unproven incumbent for another while also yielding markedly worse descriptions by the paper's own metrics — on AwA2, TC = 0.3604 at ten tags per cluster against TC = 0.6904 at 4.48 — so the certified convention is both the defensible and the more informative choice. Appendix A.3 records the provenance of the shipped values.

C.6. Defect Inventory

Six defects were identified across the two codebases: the pipeline that produced the results of Chapter 4, and the reproduction of Chapter 5. They are listed here because two of them affected numbers that were at one point reported, and because the remainder bear on how the ILP results in both chapters should be read.

1. **Hardcoded solver status.** An `"optimal"` status was recorded without being read back from the solver. Reproduction only.
2. **Solution status never inspected.** The solver's solution status was not read, so a time-limited incumbent could not be distinguished from a proven optimum. Both codebases. This defect produced incorrect reported figures.
3. **Timeout read as infeasibility.** A time limit reached without an incumbent was treated as proof of infeasibility, causing the search to advance to the next β and report an inflated value. Both codebases. This defect produced incorrect reported figures.
4. **Mismatched save state.** The saved allocation matrix W was paired with cluster memberships from a later point in training, so reported TC and ITF could mix two training states. Reproduction only; visible as a disagreement between the recorded active-cluster counts.
5. **Hidden α-infeasibility.** Infeasibility caused by the coverage parameter was rejected per β without being distinguished from infeasibility caused by the orthogonality bound, making an unsatisfiable α indistinguishable from an inert description channel. Both codebases. This is what initially masked the finding of Section 5.6.1.
6. **Silent greedy fallback.** A heuristic allocation could be substituted for the ILP solution without the substitution being recorded. Present in the main pipeline only, and confirmed never to have triggered in any logged run.

Defects 2 and 3 are the two that produced wrong numbers, and Section C.5 reports what re-solving established. The defects of the earlier trained attempts are separate and remain documented in Appendix A.2.

C.7. Runtimes

All reproduction runs were executed on the same RTX 4090 machine used for the experiments of Chapter 4. Runtimes are per seed and include ILP solving.

On AwA2, a run at $\mu = 1.5$ with the ILP enabled took 529 seconds, against 58 seconds with the ILP disabled; the paper-faithful $\mu = 1.0$ configuration took 615 seconds

and the $\lambda = 0$ configuration 721 seconds. Raising the solver limit from 20 to 120 seconds per β raised the total to between 50 and 61 minutes per seed. The 89% ILP share quoted in Section 5.6.2 is the difference between the first two of these figures.

On aPY-15 the cost is dominated by how often the ILP is feasible. At $\alpha = 1$ a run takes 14 to 21 seconds; at $\alpha = 2$, where the problem is feasible in nearly every round but harder, 269 to 705 seconds; at $\alpha = 3$, 13 to 311 seconds, scaling with the number of rounds in which a solution exists; and at $\alpha = 4$ or with the ILP disabled, 13 to 16 seconds. The k-means baselines took approximately two minutes for four configurations across five seeds. Recovering the per-instance annotations takes about two seconds of CPU time.

C.8. Code Inventory

The reproduction is separate from the pipeline of Chapter 3 and writes only to `results_v2/`.

- `ddc_v2/data.py` — dataset construction, feature caching, tag loading and masking.
- `ddc_v2/model.py` — the three-layer encoder with softmax output.
- `ddc_v2/objectives.py` — mutual information objective, pair generation from Equation (5), pairwise loss, ILP solve and mask construction.
- `ddc_v2/train.py` — pretraining and round loop, checkpointing, metric logging.
- `run_ddc_v2.py` — command-line entry point, configuration handling, and a write guard preventing any output under `results/`.
- `kmeans_sanity_baseline.py` — the matched k-means baselines of Tables C.1 and C.2.
- `build_apy_instance_tags.py` — recovery and verification of the aPY per-instance annotations (Section C.4).
- `ilp_probe.py` — re-solves a stored assignment at an arbitrary time limit and records the solver's real termination reason; used for the audit of Section C.5.
- `v1_certify.py` — reconstructs the shipped seed-42 clusterings from cached features, verifies them against the stored artefacts, and emits probe-compatible inputs.